

Harness Engineering： 从 Prompt 到可进化 Agent 系统

副标题：可运行、可观测、可验证、可持续改进的 Agent 工程实践

作者：大铭

联系邮箱：yinwm@outlook.com

微信二维码



扫一扫上面的二维码图案，加我为朋友。

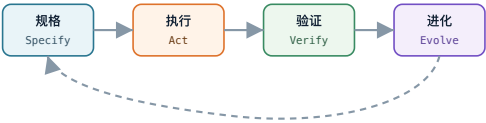
定位：面向工程师、产品负责人和团队管理者的连续阅读型小册子 阅读方式：先读第 1 至 7 章建立主线，再按自己的场景进入第 8 至 30 章实践。

这本小册子适合连续阅读。它把 Harness Engineering 当作一套可落地的 Agent 工程系统来讲：从任务规格、上下文、工具、权限，到验证、可观测性和受控自进化，目标是帮助

你把 Agent 从“不稳定的会话能力”变成“可运行、可复盘、可持续改进的系统能力”。

Harness Engineering

从 Prompt 到可进化 Agent 系统



可运行 · 可观测 · 可验证 · 可持续改进

目录

1. 这本小册子的主线：Harness 是可进化系统
2. 一页理解 Harness Engineering：从执行闭环到进化闭环
3. 为什么 Prompt Engineering 不够：它不能保存和验证进化
4. Agent 到底在什么系统里工作：从 Agent Loop 到 Evolution Loop
5. Harness 的核心定义：受控自进化系统
6. 自进化总模型：让 Harness 从失败中长出来
7. Harness 的九层结构：九个可进化面
 1. 第一层：任务规格
 2. 第二层：上下文
 3. 第三层：工具
 4. 第四层：执行环境和权限
 5. 第五层：记忆
 6. 第六层：技能
 7. 第七层：工作流和协议
 8. 第八层：验证和评测

6. 第九层：可观测性和反馈回路
 7. Human-in-the-loop：人应该在哪里介入
 8. 成本、缓存和速度
 9. Coding Agent Harness
 - !0. Research Agent Harness
 - !1. Browser / UI Agent Harness
 - !2. Data Agent Harness
 - !3. Long-running Agent Harness
 - !4. Multi-agent Harness
 - !5. 从零设计一个自进化 Harness
 - !6. 如何诊断 Harness：从失败定位到进化候选
 - !7. Harness 成熟度模型：从静态可用到自进化
 - !8. 常见反模式：伪自进化、规则堆积和漂移
 - !9. 七天学习和实践路线：每天建立一个进化闭环
 - !0. 模板库：把经验变成可进化资产
 - !1. 术语表
 - !2. 自进化资料地图：论文、系统文章和实践项目
 - !3. 推荐阅读：按进化主线学习 Harness
-

1. 这本小册子的主线：Harness 是可进化系统

本章学习目标

本章先建立全书主线：**Harness** 是一个让 **Agent** 从真实运行中持续变好的工程系统。

这意味着你读这本小册子时，不应该只问：

我怎样让 Agent 本次任务做对？

还要问：

如果这次没做对，系统怎样吸收这个失败，让下次更接近正确？

这就是本书的核心判断。

为什么把自进化放在主线里

如果把“自进化”放在最后，它很容易被理解成一个高级附加能力：先有普通 Harness，再加一个学习模块。

但更准确的理解是：自进化不是外接插件，而是 Harness 的主线。任务规格、上下文、工具、权限、记忆、技能、工作流、验证、可观

测性，这些层不是静态组件，而是会随着任务、失败、反馈和评测不断变化的资产。

因此，本书采用三个组织原则：

1. 把自进化前置到核心定义之后、九层结构之前。
2. 把九层结构重新解释为“九个可进化面”。
3. 把诊断、成熟度、反模式、七天路线和模板库都改成服务于进化闭环。

本书的核心判断

全书有一个贯穿始终的判断：

Harness Engineering 的高级形态，不是一次性写出完美规则，而是建立一个能从证据中改进、从失败中升级、从过期经验中清理自己的系统。

这套系统至少要有五个能力：

能力	含义
捕获证据	保存输入、过程、工具、输出、验证和反馈
定位失败	判断问题发生在哪一层 Harness
生成候选	把失败变成可执行的改进方案
验证改进	用 eval、golden task、人工审查或真实任务回放检查改动
推广或回滚	通过的进入正式 Harness，失败的撤回或归档

没有这五个动作，所谓“自进化”很容易变成一句空话。

从“会用 Agent”到“会经营 Agent 系统”

会用 Agent 的人，会写 prompt。

会管理 Agent 的人，会设计任务边界、工具权限、验证流程、日志和反馈。

会经营 Agent 系统的人，还会让这些边界、权限、流程、日志和反馈随着真实工作不断更新。

三者的差别很大。

会用 Agent 的人通常问：

- 我该怎么问它？
- 有没有更好的提示词？
- 换哪个模型更强？

会管理 Agent 的人会问：

- 这个任务的验收标准是什么？
- Agent 需要看到哪些上下文？
- 哪些工具可以自动调用，哪些需要批准？
- 出错后怎么知道错在什么环节？

会经营 Agent 系统的人会继续追问：

- 这次失败应该改变哪一层 Harness？
- 这个改变如何验证？
- 它应该进入 memory、skill、eval、workflow，还是 tool wrapper？
- 这个经验什么时候会过期？
- 如果改坏了，如何回滚？

整本书就是围绕最后这组问题展开。

本书的核心假设

这本小册子有四个基本假设。

第一，模型会继续变强，但不会消除工程系统的必要性。模型越强，能做的事越多，错误的影响范围也越大。

第二，真实任务不是一次问答，而是多步行动。只要涉及工具、文件、网络、数据库、浏览器、代码、业务判断，就需要 Harness。

第三，Harness 不应该是静态文档。静态文档会过期，过期规则会误导 Agent。好的 Harness 要有更新、验证和清理机制。

第四，自进化必须受控。没有评估门槛、权限边界、版本记录和回滚机制的自进化，不是进化，而是漂移。

读完本书你应该得到什么

读完本书，你应该能独立完成三件事：

1. 为一个真实 Agent 任务设计 Harness。
2. 为这个 Harness 建立最小进化闭环。
3. 判断一次失败应该转化为 memory、skill、eval、tool wrapper、workflow 还是人工规则。

这本小册子不是让你多知道一个流行词，而是让你把 Agent 从“能做事”推进到“在可控系统里持续做得更好”。

2. 一页理解 Harness Engineering：从执行闭环到进化闭环

本章主线：这一章不再只解释 Harness 是什么，而是先建立一个判断：Harness 的价值在于让 Agent 的每次运行都能变成下一版系统的证据。

本章学习目标

这一章要形成一张脑图：Agent 是执行者，Harness 是工作系统。不要把 Harness 想成某个库，也不要想成一个固定产品。它是一组工程结构。

更具体的比喻

你可以用三个比喻理解 Harness。

第一个比喻是汽车安全系统。

模型像发动机，工具像轮胎和方向盘。

Harness 则包括安全带、刹车、仪表盘、导航、限速、碰撞预警和维修记录。发动机越强，越需要这些系统。

第二个比喻是工厂生产线。

Agent 能做某道工序，但生产线决定原料怎么进入、工序如何衔接、质量怎么检查、坏品怎么返工、产能怎么提升。

第三个比喻是新人入职手册加工作流。

你不会把一个新人丢进公司，然后只说“把业务做好”。你会给他项目文档、权限、工具、验收标准、代码审查、导师和反馈。Agent 也一样。

Harness 的最小闭环

最小 Harness 可以非常简单：

任务规格

- > Agent 执行
- > 自动验证
- > 输出结果
- > 记录失败
- > 更新规则

这个闭环里最重要的是后两步。很多人会让 Agent 执行，也会看结果，但不会把失败转化为系统改进。这样每次都是重新开始。

判断一个系统是不是 Harness

可以用五个问题判断：

1. 它是否明确告诉 Agent 目标和边界？

2. 它是否控制 Agent 能看什么和做什么？
3. 它是否自动检查一部分结果？
4. 它是否记录过程和失败？
5. 它是否让下一次同类任务更稳定？

如果五个问题都是否定，那它只是一次 Agent 调用。

如果前两个是肯定，它是初级 Harness。

如果五个都是肯定，它已经有闭环。

先给一个最短定义：

Harness Engineering 是围绕 AI Agent 设计、实现和持续改进其运行环境的工程实践。

这里的“运行环境”不是单纯的服务器或容器，而是一个更大的系统：

用户目标

|

v

任务规格 -> 上下文 -> 工具 -> 执行环境 -> 验证 -> 反馈
-> 记忆/规则更新

|

^

+----- 人类审查与接管 -----

-----+

模型只是这个系统的一部分。一个强模型放在混乱环境里，仍然会做出不稳定结果。一个普通模型放在设计良好的 Harness 里，可能比强模型裸跑更可靠。

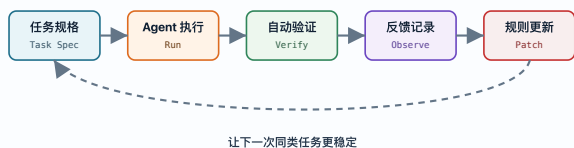
你可以把 Harness 理解成三个东西的组合：

1. 轨道：告诉 Agent 应该沿什么路径工作。
2. 护栏：限制 Agent 不该做什么。
3. 仪表盘：让人和系统知道 Agent 做了什么、哪里错了、怎么改进。

在传统软件工程里，我们不会只靠程序员“凭感觉写代码”。我们有需求文档、接口协议、测试、CI、日志、监控、回滚、权限、代码审查。Agent 进入工程世界后，也需要类似的一套系统。Harness Engineering 就是这套系统的设计方法。

Harness 最小闭环

一次任务不只是输出结果，还要留下可改进的证据。



3. 为什么 Prompt Engineering 不够：它不能保存和验证进化

本章主线：Prompt Engineering 解决“本次怎么说”，Harness Engineering 解决“下次系统如何变好”。如果一个经验不能被保存、验证和复用，它就没有进入 Harness。

本章学习目标

这一章要拆掉一个误解：Agent 失败时，第一反应不应该总是“prompt 写得不够好”。

Prompt 当然重要，但 prompt 很难独自承担上下文管理、权限控制、状态保存和结果验证。

Prompt 能解决什么

Prompt 擅长解决：

- 角色设定。
- 输出格式。
- 任务偏好。
- 思考步骤提示。

- 风格要求。
- 局部约束。

例如：

请用中文输出。
先列风险，再给方案。
不要改 public API。
输出 Markdown 表格。

这些都是 prompt 的强项。

Prompt 不能稳定解决什么

Prompt 不擅长稳定解决：

- 工具实际是否可用。
- Agent 是否真的跑了测试。
- Agent 是否读到了最新文件。
- Agent 是否有权限执行某个命令。
- Agent 是否泄露了不该泄露的数据。
- Agent 是否把失败记录下来。
- Agent 是否下次还记得这次教训。

这些需要系统结构。

一个反例

你可以写一个很长的 prompt：

你是一个资深工程师。请认真阅读代码，谨慎修改，运行测试，避免破坏现有功能，遵循最佳实践，保持代码整洁。

这看起来很专业，但它没有回答：

- 读哪些代码？
- 哪些文件不能改？
- 测试命令是什么？
- 测试失败怎么办？
- 什么叫最佳实践？
- 如何证明没有破坏现有功能？

所以它只能提升表达质量，不能保证工程质量。

从 prompt 到 Harness 的升级

把一句 prompt 升级成 Harness，需要把隐含要求变成可执行结构。

隐含要求	Harness 化
认真阅读代码	指定 repo map 和必读文件
谨慎修改	限制 write scope
运行测试	固定 verification command
不破坏现有功能	回归测试和 diff review
遵循最佳实践	项目规则和代码审查清单
保持整洁	formatter、lint、review rubric

Prompt 是自然语言愿望。Harness 要把愿望变成流程、工具和检查。

Prompt Engineering 解决的是“怎么把话说清楚”。这当然重要，但它只能覆盖很小一部分问题。

假设你对 Agent 说：

请帮我修复这个 bug，保持代码风格一致，运行测试，不要破坏现有功能。

这句话看起来很合理，但真实执行时会遇到很多问题：

- Agent 不知道项目真实架构。
- Agent 不知道哪些文件不能动。
- Agent 不知道测试怎么跑。
- Agent 不知道失败日志是什么意思。
- Agent 不知道哪些行为需要用户批准。
- Agent 可能修了 bug，但引入新问题。
- Agent 可能测试没有跑完就说完成了。
- Agent 可能把一次性经验忘掉，下次又犯同样错误。

这些问题不是靠多写几句 prompt 就能稳定解决的。你真正需要的是：

- 项目地图。

- 任务边界。
- 工具协议。
- 权限系统。
- 验证步骤。
- 日志和审计。
- 经验沉淀。

Prompt 是 Harness 的一个入口，不是 Harness 本身。

可以用下面的表区分：

层次	主要问题	典型产物
Prompt Engineering	怎么指挥模型	prompt、system message、few-shot example
Context Engineering	给模型看什么	文件摘要、检索结果、知识片段、运行状态
Tool Engineering	模型能调用什么	CLI、API、browser、database、MCP tools
Eval Engineering	怎么判断好坏	tests、benchmarks、rubrics、LLM judge
Harness Engineering	整个 Agent 如何稳定工作	任务协议、上下文、工具、权限、日志、评测、反馈闭环

Prompt Engineering 像是“说明书”。Harness Engineering 像是“生产线”。

4. Agent 到底在什么系统里工作：从 Agent Loop 到 Evolution Loop

本章主线：Agent Loop 只描述一次执行；Evolution Loop 描述多次执行之间如何积累、归因、修正和升级。理解这个差别，是理解全书的关键。

本章学习目标

本章要让你理解 Agent loop 的风险。只要 Agent 能调用工具，它就不是静态文本生成器，而是一个会行动的系统组件。

Agent loop 的真实展开

一个 Agent loop 往往不是简单的“想一下，做一下”。真实过程更像：

读取任务

- > 解释目标
- > 选择上下文
- > 读文件或网页
- > 建立计划
- > 选择工具
- > 执行动作
- > 观察输出
- > 判断是否继续

- > 修改计划
- > 再行动
- > 形成结果

每一步都可能出错。

每一步的典型错误

步骤	典型错误
解释目标	把用户没说的东西当成目标
选择上下文	漏掉关键文件，读到过期文档
建立计划	计划太大，无法验证
选择工具	用错工具，或使用高风险工具
执行动作	改动范围过大，命令跑错目录
观察输出	忽略错误日志，只看最后一行
判断继续	没完成就停止，或陷入循环
形成结果	报告过度自信，隐瞒未验证项

Harness 的作用就是在这些节点上加结构。

为什么 Agent 比脚本更需要 Harness

传统脚本通常路径固定：

输入 -> 规则 -> 输出

Agent 的路径是动态的：

输入 -> 推理 -> 选择动作 -> 观察 -> 再推理 -> 再选择动作

动态路径更灵活，但也更难预测。所以 Agent 需要更多观测和约束。

关键判断

如果一个任务只有一次文本输出，Harness 可以很轻。

如果一个任务涉及多步行动，Harness 必须更重。

如果一个任务涉及写入、外部发送、金钱、隐私、生产系统，Harness 必须有权限和审计。

理解 Harness 之前，要先理解 Agent 和普通 LLM 调用的差别。

普通 LLM 调用大致是：

输入 -> 模型 -> 输出

Agent 调用更像：

目标

|

v

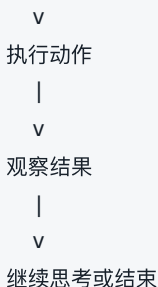
模型思考

|

v

选择工具

|



这就是 Agent loop。

Agent loop 一旦进入真实世界，就会放大很多问题：

- 模糊目标会变成随机决策。
- 错误上下文会引导 Agent 走错路。
- 工具太强会带来破坏性风险。
- 工具太弱会让 Agent 绕远路。
- 没有验证会导致假完成。
- 没有日志就无法复盘。
- 没有记忆就会反复犯错。
- 没有成本控制就会烧 token 和时间。

所以，Agent 不是“一个会说话的函数”。

Agent 更像一个初级同事、自动化脚本和搜索系统的混合体。你必须给它工作环境、权限、流程和检查机制。

5. Harness 的核心定义：受控自进化系统

本章主线：在这个定义里，Harness 不只是围绕 Agent 的脚手架，而是一个受控自进化系统：它捕获证据、定位失败、提出改进、经过验证后推广。

本章学习目标

这一章要把 Harness 从抽象名词拆成可执行动作。你要能看到一个 Agent 系统，然后指出它的 Harness 在哪里，缺了哪些部分。

八个动作的详细解释

1. Specify：定义任务

Specify 是把“帮我做一下”变成“做到什么算完成”。没有 Specify，Agent 会替你做产品经理。

例子：

差：帮我写个报告。

好：基于 10 个指定来源，写一份 3000 字中文报告，必须区分事实、观点和推断，最后给出 5 条可执行建议。

2. Ground: 建立上下文

Ground 是让 Agent 站在正确事实上。错误上下文会导致漂亮但错误的输出。

例子：

不要只告诉 Agent “这是个 Go 项目”。

要告诉它入口命令、测试方式、核心目录、哪些文件是生成物、哪些规则不能破坏。

3. Equip: 提供工具

Equip 是给 Agent 行动能力。没有工具，Agent 只能建议。工具太强又没有限制，会造成风险。

4. Constrain: 约束行为

Constrain 是告诉 Agent 不能做什么。工程里很多重要规则是负面边界：

- 不要改 public API。
- 不要提交代码。
- 不要打印密钥。
- 不要删除用户数据。
- 不要把私有聊天当公开来源。

5. Orchestrate: 安排流程

Orchestrate 是控制顺序和角色。复杂任务需要拆阶段，而不是让 Agent 一口气做完。

6. Verify: 验证结果

Verify 是把“看起来完成”变成“有证据完成”。

7. Observe: 观察过程

Observe 是记录 Agent 做了什么。没有观察，失败无法复盘。

8. Improve: 持续改进

Improve 是 Harness 和普通流程的区别。失败不只是修复当前任务，还要进入下一轮系统。

Harness 的边界

Harness 不一定包含模型。一个 Harness 可以适配多个模型。

Harness 不一定是代码。早期可以是 Markdown 规则和 checklist。

Harness 不一定很重。只要有闭环，轻量 Harness 也有价值。

本小册子采用这个定义：

Harness 是一组围绕 Agent 的工程结构，用来约束输入、组织上下文、提供工具、控制权限、执行任务、验证结果、记录过程，并把失败转化为下一轮改进。

拆开来看，Harness 至少包含八个动作：

1. Specify: 把任务变成可执行规格。
2. Ground: 把 Agent 放到正确上下文里。
3. Equip: 给 Agent 合适工具。
4. Constrain: 限制权限和行动范围。
5. Orchestrate: 安排执行顺序和角色。
6. Verify: 检查结果是否正确。
7. Observe: 记录过程和失败。
8. Improve: 把经验转化为规则、测试、技能或记忆。

这八个动作形成闭环：

规格 -> 执行 -> 验证 -> 观察 -> 改进 -> 新规格

如果一个系统只有规格和执行，没有验证和改进，那它不是完整 Harness。

如果一个系统只有工具，没有权限和日志，那它只是工具箱。

如果一个系统只有规则，没有反馈，那它只是文档。

如果一个系统能持续从失败中变好，它才接近 Harness。

6. 自进化总模型：让 Harness 从失败中长出来

本章学习目标

自进化是 Harness Engineering 里最重要、也最容易被误解的一部分。它听起来像一句口号，但落到工程上，它不是“模型突然有了生命力”，也不是“Agent 说自己学到了”。它指的是一个可追踪、可验证、可回滚的系统改进闭环。

读完本章，你要能区分两件事：

1. 真正的自进化：系统从真实运行、失败、评测和人工反馈中改进自己的规则、工具、技能、上下文和 eval。
2. 伪自进化：Agent 在一次对话里说“我学到了”，但没有任何规则、代码、测试、记忆或流程被改变。

自进化不是玄学，也不是让 Agent 完全自动改自己。更准确的说法是：

自进化是一个受控的 Harness 改进闭环：每次执行产生证据，证据暴露失败，

失败触发改进候选，改进经过验证后进入下一版 Harness。

这章会回答四个问题：

1. Harness 到底能进化什么。
2. 哪些进化可以自动化，哪些必须人工审查。
3. 如何把失败变成 eval、memory、skill、tool 或 workflow 的改进。
4. 如何避免自进化变成自我污染。

为什么 Harness 必须自进化

Agent 系统面对的是变化环境：

- 模型会升级。
- 工具会变化。
- 项目结构会变化。
- 用户偏好会变化。
- 任务类型会变化。
- 失败模式会变化。

如果 Harness 不进化，它会逐渐过期。

过期的 Harness 会出现这些症状：

- 规则越来越长，但命中率越来越低。

- Agent 经常被旧约束误导。
- 新模型已经能做的事，旧流程还在强行拆碎。
- 旧 eval 只覆盖过去的问题，不能发现新问题。
- memory 里堆满过时经验。
- skill 文档写得很完整，但没人再用。

所以，Harness 不是写完就结束的文档，而是一个需要维护的系统。

一个现实判断是：**静态 Harness 最多只能覆盖过去的失败；自进化 Harness 才能吸收未来的失败。**

这也是为什么“自进化”不应该放在小册子的边角。对于个人使用者，它决定 Agent 会不会越用越顺手。对于团队，它决定 Agent 系统会不会随着项目积累经验。对于组织，它决定治理规则、验证标准和工具链能不能跟上真实业务变化。

自进化的对象

Harness 可以进化的不是一个东西，而是一组组件。

组件	可以怎么进化
Task Spec	增加更清晰的验收标准、风险边界、非目标
Context Pack	增加必读材料，移除过期上下文，重排优先级
Tools	增加 wrapper，改输出格式，限制高风险参数
Permissions	调整审批策略，给低风险动作放权，收紧高风险动作
Skills	把重复流程固化成技能，给技能加样例和失败处理
Memory	保存稳定经验，删除过期记忆，补充来源
Workflow	增加检查点，拆阶段，明确 handoff
Eval	从失败样本里增加 golden task，调整评分规则
Observability	增加日志字段，记录更多可复盘信息
Agent Topology	调整单 Agent、多 Agent、审查 Agent、执行 Agent 的分工
Prompt / System Context	重写指令、重排上下文、拆成可复用 playbook
Code / Scaffolding	改进脚手架、脚本、hook、validator、wrapper

自进化不是只改 prompt。只改 prompt 是最轻的一层，很多时候不够。

可以把 Harness 的可进化对象分成三层：

浅层进化: prompt、checklist、memory、few-shot 示例
中层进化: skill、workflow、eval、tool wrapper、权限策略
深层进化: agent architecture、meta-agent、代码脚手架、自动设计空间

越往深层，收益越大，风险也越大。浅层进化可以快，中层进化要有 review，深层进化必须有 sandbox、版本控制、回滚和评估集。

自进化的四种主流路线

英文资料和论文里，“self-improving”“self-evolving”“recursive self-improvement”“agentic evolution”经常混用。为了做 Harness，不需要一开始陷进名词里。可以按工程机制分成四条路线。

路线一：反思式进化

代表思路是 Reflexion、Self-Refine、CRITIC、Self-Debugging。

核心循环是：

尝试 -> 得到反馈 -> 反思错误 -> 改写答案或下一步策略

这条路线对 Harness 的启发是：失败不能只保存在聊天记录里，而要进入 episodic memory、failure review 或 eval seed。它适合解决“同一类任务下次少犯错”的问题。

路线二：上下文进化

代表思路是 Agentic Context Engineering。

核心循环是：

运行轨迹 -> 提取经验 -> 生成候选上下文 -> 反思和整理 -> 更新 playbook

这条路线对 Harness 最直接。很多个人和团队并不会训练模型，而是通过系统提示、项目规则、memory、技能和上下文包让 Agent 变强。上下文进化就是把这些材料从“越堆越乱”改成“结构化长大”。

路线三：技能和工作流进化

代表思路是 self-improving-agent、Evolver、wow-harness 里的 failure pattern extraction、skill growth、validator growth。

核心循环是：

重复失败或重复任务 -> 抽象模式 -> 固化为 skill / validator / workflow -> 下次自动加载

这条路线最适合做个人和团队 Harness。它不要求模型权重变化，也不要求复杂 RL。只要有运行日志、失败记录、模板和验证，就能开始。

路线四：架构和程序进化

代表思路是 Promptbreeder、STOP、ADAS、Gödel Agent、Darwin Gödel Machine、Hyperagents、AlphaEvolve。

核心循环是：

```
候选 agent/program -> 自动评估 -> 保留高分版本 -> 生成新变体 -> 继续搜索
```

这条路线很强，但不能直接照搬到普通业务系统。它的前提是有明确 evaluator，最好有 sandbox 和可重复 benchmark。没有 evaluator 的“自动改自己”，不是自进化，是高风险漂移。

自进化的工程定义

在 Harness Engineering 里，可以采用一个更窄、更实用的定义：

```
Self-evolving Harness =  
  Traceable experience capture  
  + failure attribution  
  + candidate change generation  
  + gated verification  
  + promotion / rollback
```

每个词都很关键。

Traceable experience capture 意味着系统要保存足够证据：输入、输出、工具调用、失败现象、验证结果、用户纠正。

failure attribution 意味着系统要判断错在哪一层。是任务规格不清，还是上下文缺失，还是工具失败，还是权限不足，还是 eval 漏检。

candidate change generation 意味着系统要提出具体改动，不是写一句“下次注意”。

gated verification 意味着改动必须被测试。测试可以是自动 eval，也可以是人工审查加 golden task。

promotion / rollback 意味着通过的改动进入正式 Harness，失败的改动撤回或留在候选区。

这一定义有一个直接推论：

没有评估门槛的自进化，不叫进化，叫漂移。

自进化闭环

一个可操作的自进化闭环可以分成八步：

1. Run

执行一次真实任务。

2. Observe

记录输入、工具、过程、输出、失败和验证结果。

3. Diagnose

判断失败发生在哪一层：规格、上下文、工具、权限、流程、验证、记忆、模型。

4. Propose

提出 Harness 改进候选。

5. Patch

修改规则、模板、工具、skill、eval 或 memory。

6. Verify

用同类任务或 golden task 检查改动是否有效。

7. Promote

通过验证后，把改动升级为正式 Harness 版本。

8. Prune

删除过期规则、重复 memory 和无效 skill。

这八步里，最容易缺的是 Verify 和 Prune。

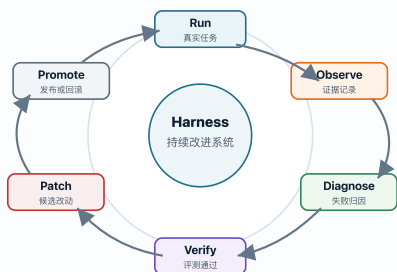
没有 Verify，系统不知道改动是否真的有效。

没有 Prune，Harness 会越来越臃肿。

可以把它画成一个闭环：

受控自进化闭环

进化不是让系统随意自改，而是让每个改进经过证据、验证和回滚边界。



自进化和自动进化的区别

自进化不等于全自动进化。

可以分成四级：

Level A：人工驱动进化

Agent 记录失败，人类决定怎么改 Harness。

适合早期。

Agent 输出失败复盘

人类决定是否改规则

人类手动更新文档或工具

Level B：Agent 提议，人类批准

Agent 根据失败提出改进候选，人类审查后批准。

适合中期。

Agent：我建议把“URL 展开失败要标记原因”加入 Research Protocol。

Human: 同意。

System: 更新 protocol, 并加入 eval。

Level C: 低风险受控自动进化

低风险改动可以自动进入候选分支或草稿区, 通过 eval 后自动合并。

适合成熟系统。

失败样本 -> 自动生成 eval -> 跑 regression -> 通过后更新 skill patch

Level D: 开放式架构进化

系统不只更新规则, 还会搜索新的 agent 架构、工具组合、工作流和程序实现。

适合研究系统、强评估场景、代码竞赛、算法发现、受限 sandbox。

agent variant archive -> generate new variant -> evaluate -> retain -> mutate again

不要一开始就追求 Level C 或 Level D。没有 eval 和版本控制, 自动进化很容易变成自动污染。

Harness 自进化的分级审批

自进化要做得稳, 必须给不同改动设置不同审批级别。

改动类型	风险	默认审批
添加失败记录	低	自动
添加 memory candidate	低	自动进入候选区
更新 checklist	中低	Agent 提议，人类快速确认
添加 eval case	中	自动生成，人工抽查
更新 skill 行为	中高	人类审查
修改工具 wrapper	高	测试通过后合并
放宽权限	高	必须人工批准
自动执行新 shell 命令	高	必须人工批准
修改 agent 架构或 orchestration	高	评审 + regression
删除旧规则	中	需要 prune reason

一个好的 Harness 不是“能自动改一切”，而是知道什么可以自动，什么必须停下来。

自进化的输入

系统从哪里获得进化信号？

1. 失败日志

例如：

报告漏掉了本地出现次数最高的 URL。

可能改进：

- 增加 URL frequency coverage eval。
- 在 Research Workflow 里加入 top URL 展开步骤。

2. 用户纠正

例如：

用户说：不是要基于资料收集工具写报告，工具只是背景。

可能改进：

- 更新 Task Spec 理解规则。
- 在报告生成前增加“确认主题和工具边界”检查。

3. Eval 失败

例如：

Privacy check failed: output contains private attachment URL.

可能改进：

- 增加 private URL regex。
- 更新输出前 checklist。

4. 工具错误

例如：

Feishu doc create failed because app lacks docx:create permission.

可能改进：

- 在 Feishu export workflow 中先做 permission preflight。
- 增加 fallback：生成 docx import file。

5. 成本指标

例如：

每次研究都重复展开同一批 URL。

可能改进：

- 增加 source expansion cache。
- 对已展开来源做 hash 和更新时间。

6. 用户满意度和人工介入

例如：

同一类任务里，用户每次都要补一句“展开 URL，不要只列链接”。

可能改进：

- 在 Research Harness 里把 URL 展开变成强制步骤。

- 给最终报告增加 source coverage checklist。

7. 运行轨迹中的重复模式

例如：

每次创建飞书文档都先失败一次，然后才生成 docx fallback。

可能改进：

- 在 workflow 开头加入 permission preflight。
- 把 fallback 写成正式路径，而不是临时补救。

8. 成功案例

自进化不只从失败中学，也可以从成功中学。

例如：

某次报告结构被用户明确认可。

可能改进：

- 提炼为 handbook writing template。
- 加入“课程型文档必须包含解释、例子、反例、练习、模板”的写作 skill。

自进化记录卡片

每次 Harness 进化都应该有记录。

Evolution Record

Date

YYYY-MM-DD

Trigger

User corrected the task framing: the data-gathering tool is only background, not the report topic.

Failure Layer

Task Spec / Context Framing

Evidence

The generated report over-centered the tool instead of the topic.

Change Proposed

Add a pre-report framing check:

- What is the real topic?
- What is only a tool?
- What should not become the main narrative?

Change Applied

Updated Research Harness workflow.

Verification

New handbook version centers Harness Engineering, with the data-gathering tool mentioned only as background.

Status

Promoted

这种记录让 Harness 的进化可追踪，而不是靠记忆。

更成熟的记录要包含 rollback 条件：

Evolution Record

ID

EV-YYYY-MM-DD-001

Trigger

The user asked for a learning handbook, but the output was closer to a research memo.

Evidence

- User correction: "我要学习 Harness, 工具只是背景"
- Missing elements: examples, anti-examples, exercises, source map

Failure Layer

Task Spec / Genre Recognition / Teaching Depth

Proposed Change

Add a Long-form Learning Document protocol:

- identify target genre
- separate topic from tool
- expand each concept with explanation, example, anti-example, checklist, exercise

Patch Scope

- Research workflow
- Writing checklist
- Handbook eval

Verification

Run the next handbook chapter against:

- concept clarity
- examples
- anti-examples
- actionable template
- source map

Promotion Criteria

The user can read the chapter as a course, not as a chat summary.

Rollback Criteria

If the protocol causes short answers to become bloated, restrict it to "booklet/course/long-form guide" triggers.

Status

Candidate / Promoted / Rolled back

自进化中的 eval 生成

成熟 Harness 会从失败中生成 eval。

例子：

失败：

报告太像研究记录，不像学习课程。

新增 eval：

Eval: Handbook Depth

Input:

Generated handbook chapter.

Checks:

- Does the chapter explain the concept?
- Does it include example?
- Does it include anti-example?
- Does it include practical checklist or exercise?
- Can a reader apply it after reading?

Fail if:

- The chapter only gives definitions.

- The chapter has no concrete scenario.
- The chapter has no action step.

这样，下次生成学习材料时，系统不会只追求长度，而会检查教学深度。

Eval 不是越多越好

很多团队一听 eval，就会开始堆测试。自进化里的 eval 要避免三个问题：

1. 只测容易自动化的部分，不测真正关键的部分。
2. 只测过去的失败，不保留通用能力样本。
3. 用同一个模型既生成改进又给改进打分，没有外部信号。

更可靠的做法是保留三类 eval：

Eval 类型	用途
Regression eval	确认旧能力没有退化
Failure-derived eval	确认同类失败被修复
Generalization eval	确认新规则没有过拟合

举例：

失败：报告没有展开 URL。

Failure-derived eval：

- 输入包含 10 个 URL。
- 输出必须区分：已展开、无法访问、低相关。

Generalization eval:

- 输入包含 3 个 URL 和 2 个本地文件。
- 输出不能只处理 URL，也要处理本地证据。

Regression eval:

- 输入是纯本地文件任务。
- 输出不应该强行联网。

自进化中的 memory 管理

自进化离不开 memory，但 memory 也最容易污染。

Memory 进入系统前要问：

- 这是稳定规律，还是一次临时反馈？
- 是否会影响未来任务？
- 是否有来源？
- 是否含隐私？
- 什么时候应该失效？

可以给 memory 加生命周期：

```
## Memory Candidate
```

Content:

When creating a learning handbook, include
explanations, examples, anti-examples,
exercises, and templates.

Source:

User feedback on Harness handbook.

Scope:

Writing educational documents.

TTL:

Long-term, review after 3 months.

不是所有反馈都应该永久记忆。

Memory 的四个状态

可以把 memory 分成四个状态：

状态	含义
Candidate	刚从失败或反馈中提取，还没验证
Active	已验证、会被下次任务加载
Scoped	只在某个项目、某类任务或某个用户偏好里生效
Archived	过期、被替换或暂时不用

不要让 Candidate 直接变成全局规则。很多污染就是这样发生的。

Memory 的最小字段

Memory Item

Statement

When the user asks for a booklet/course, produce a teaching artifact, not only a research report.

Scope

Long-form educational writing.

Evidence

User corrected the Harness booklet direction on 2026-05-19.

Confidence

High

Expiry / Review

Review after 3 months or if user asks for concise mode.

Status

Active

自进化中的 skill 更新

Skill 更新要比 memory 更严格，因为 skill 会改变执行过程。

Skill 更新流程：

发现重复任务

- > 写出当前人工步骤
- > 抽象输入输出
- > 写成 skill
- > 用两个样例测试
- > 失败后更新

示例：

Skill: Long-form Handbook Writing

Trigger:

User asks for a course, booklet, or "one article is enough" learning document.

Required structure:

- concept
- why it matters

- examples
- anti-examples
- process
- checklist
- exercise
- templates

Failure to avoid:

- Producing a research report instead of teaching material.

这就是从一次失败中长出一个 skill。

Skill 进化的判断条件

不是所有经验都应该变成 skill。满足下面三条，才值得提升：

1. 同类任务会反复出现。
2. 单靠一句记忆不够，需要步骤、模板或工具。
3. 有办法验证 skill 是否帮助了任务。

例如“生成课程型小册子”值得变成 skill，因为它包含结构判断、写作深度、章节模板、资料引用和最终交付格式。

但“某次报告标题不够好”通常只需要一个写作反馈，不必变成 skill。

Skill 的自进化结构

```
# Skill: Harness Handbook Writer
```

```
## Trigger
```

User asks for a booklet, course, handbook, or
"one document is enough".

Inputs

- topic
- audience
- existing source material
- required output format

Workflow

1. Identify whether this is teaching, research, SOP, or reference.
2. Separate the topic from tools used to collect evidence.
3. Build a chapter map.
4. For each chapter, include concept, why, example, anti-example, checklist, exercise.
5. Add source map and further reading.

Verification

- Can a new reader apply the idea after reading?
- Are sources grouped by role?
- Are examples concrete?
- Are templates reusable?

Failure Modes

- Turning the handbook into a research log.
- Listing links without explaining why they matter.
- Giving definitions without practice.

自进化中的工具改进

工具也应该进化。

例子：

飞书文档创建失败，因为权限不足。工具层可以改进为：

```
preflight_feishu_doc:
1. Check app credentials.
2. Check docx:create permission.
3. If missing, report permission URL.
4. Generate local docx fallback.
5. Save export metadata.
```

这样下次不会先跑完整流程再失败，而是在一开始就知道能不能创建。

工具进化通常比 prompt 进化更可靠，因为它把脆弱的语言约束变成确定的执行边界。

典型工具进化包括：

工具问题	工具进化
输出太长	增加 summary mode 和 raw artifact
输出含敏感字段	增加 redaction wrapper
命令容易跑错目录	wrapper 强制 <code>cwd</code>
API 权限不足才报错	preflight 检查
搜索结果不可复现	保存 query、timestamp、source snapshot
Web 页面可能变化	保存引用摘要和访问日期
多工具结果难比较	统一 envelope schema

工具进化的原则是：把重复提醒变成机械约束。

自进化中的规则清理

很多人只会加规则，不会删规则。自进化必须包含反向动作：prune。

应该删除或降级的规则：

- 过期规则。
- 从未触发的规则。
- 与新模型能力冲突的规则。
- 多条规则表达同一件事。
- 只解决一次性问题的规则。

规则清理模板：

Rule Prune Review

Rule:

Why it was added:

Last triggered:

Still relevant:
yes / no / uncertain

Action:
keep / revise / archive / delete

没有 prune，Harness 会越来越像杂物堆。

Prune 的触发时机

应该定期做 prune review，尤其在这些情况下：

- 模型升级后，旧 workaround 可能不需要了。
- 工具接口改了，旧规则变成误导。
- 项目目录重构后，旧路径规则失效。
- 用户偏好变化后，旧输出风格不适合。
- 同一件事被多个规则重复表达。

Prune 不是删除经验，而是减少噪声。被归档的规则仍然可以保留来源和历史原因，但不应该继续污染当前上下文。

自进化的指标

自进化不能只靠感觉。可以观察这些指标：

指标	含义
Repeat failure rate	同类错误是否减少
First-pass success	首次交付是否更接近要求
Verification coverage	自动检查覆盖多少关键风险
Human correction count	用户纠正次数是否下降
Run recovery time	中断后恢复是否更快
Rule hit rate	规则是否真的被使用
Eval regression count	改 Harness 是否引入新问题
Evolution lead time	从发现失败到改进生效用了多久
Candidate promotion rate	候选改进中有多少被验证通过
Prune rate	是否持续清理无效规则
Context load size	上下文是否越进化越臃肿

最有价值的指标通常不是“Agent 做了多少”，而是“同类错误有没有减少”。

一个团队可以每周看一次：

- 本周新增失败样本数
- 本周生成改进候选数
- 本周通过验证的改进数
- 本周回滚/废弃的改进数
- 重复失败是否下降
- 用户纠正次数是否下降
- 上下文体积是否增长过快

自进化的风险

1. 过拟合

Harness 可能只适应某几个样例，对新任务变差。

对策：

- 使用多样 golden tasks。
- 保留人工抽查。
- 不要为单一失败加过窄规则。

2. 自我污染

Agent 把错误结论写进 memory 或 skill。

对策：

- Memory 需要来源。
- 高影响 skill 更新需要 review。
- 未验证结论只能进 candidate，不进正式规则。

3. 规则爆炸

每次失败都加一条规则，最后系统难以理解。

对策：

- 合并同类规则。
- 定期 prune。
- 优先改流程和 eval，而不是堆文本规则。

4. 自动化越权

系统开始自动改高风险流程。

对策：

- 分级进化。
- 高风险变更必须人工批准。
- Harness 自身也要有权限边界。

5. 指标欺骗

系统可能优化 eval 分数，但真实任务变差。

对策：

- 保留人工抽查。
- 保留真实任务回放。
- 不要只依赖一个综合分数。
- 记录“用户是否还需要纠正”。

6. 上下文坍塌

反复总结 memory 和规则时，细节被越压越少，最后只剩空泛原则。

对策：

- 采用结构化增量更新，而不是每次全文重写。
- 保留原始 evidence 链接或摘录。
- 对高价值经验保留具体样例。

- 将 playbook 拆成模块，不要只维护一个大总结。

7. 安全边界漂移

系统在“提升效率”的名义下逐渐放宽权限。

对策：

- 权限放宽必须人工批准。
- 每次权限变更要有 risk note。
- 高风险工具要有审计日志。
- 自进化系统不能自动修改自己的安全边界。

自进化和 Harness 九层结构的关系

第 7 章讲过 Harness 的九层结构。自进化不是额外一层，而是穿过九层的一条反馈回路。

Harness 层	自进化问题
任务规格	任务是否经常被误解？是否需要更清晰的验收标准？
上下文	是否缺关键资料？是否上下文过载？
工具	工具失败是否重复发生？是否需要 wrapper？
执行环境和权限	审批是否太多或太少？是否有越权风险？
记忆	什么经验应该长期保留？什么应该删除？
技能	哪些重复流程应该固化成 skill？
工作流和协议	哪个阶段最容易漂移？是否需要 gate？
验证和评测	失败是否能转化为 eval？
可观测性和反馈	日志是否足够定位问题？

因此，做自进化时不要只问“改 prompt 吗”。要问：

- 这次失败暴露的是哪一层的缺陷？
- 改哪一层最稳定？
- 改哪一层风险最低？
- 改完怎么验证？

自进化最小实现

如果你现在要给一个个人 Agent 工作流加自进化，不需要复杂平台。先做四个文件：

```
HARNESS.md
run-log.md
```

```
failure-review.md
evolution-log.md
```

最小流程：

每次任务结束：

1. 记录结果。
2. 记录失败或用户纠正。
3. 判断失败层级。
4. 写一个改进候选。
5. 下次同类任务前先读 `evolution-log`。

这已经是自进化的起点。

最小目录结构

```
.harness/
  HARNESS.md
  evals/
    golden-tasks.md
    regression-checklist.md
  evolution/
    run-log.md
    failure-review.md
    evolution-events.jsonl
    candidates/
    promoted/
    archived/
  memory/
    candidate.md
    active.md
    archived.md
  skills/
    handbook-writer.md
    research-expander.md
```

最小日常流程

任务开始前：

1. 读取 HARNESS.md。
2. 读取相关 active memory。
3. 读取相关 skill。

任务结束后：

1. 写 run-log。
2. 如果失败或被纠正，写 failure-review。
3. 判断是否生成 evolution candidate。
4. 如果 candidate 高价值，补一个 eval 或 checklist。
5. 下次同类任务前加载 candidate 或 promoted 改动。

最小命令面

即使没有复杂平台，也可以把流程做成简单命令：

```
harness log-run  
harness review-failure  
harness propose-evolution  
harness verify-evolution  
harness promote-evolution  
harness prune
```

这些命令不一定真的要写成 CLI。它们代表的是系统动作。只要动作稳定存在，Harness 就开始有自进化能力。

自进化成熟度

等级	状态
E0	没有复盘，每次从头开始
E1	人工记录失败
E2	失败转化为规则或 checklist
E3	失败转化为 skill、memory 或 eval
E4	改动经过 golden task 验证
E5	低风险改进可自动生成候选并通过审查合并

不要跳级。E1 和 E2 已经能带来明显提升。

自进化案例：研究型 Harness

以一个 Research Agent 为例。

初始失败：

用户要求“用本地资料工具查线索，也可以搜英文论文”。

Agent 只做了泛泛搜索，没有把本地资料中出现过的 URL、关键词、论文附件和英文资料对应起来。

失败层级：

Task Spec：没有区分本地线索和外部资料。

Workflow：没有先做本地关键词统计，再做外部展开。

Eval：没有 source coverage 检查。

Output：没有区分“本地命中”与“主引用”。

Harness 改进：

1. Research workflow 增加 Local Evidence Pass。
2. 对本地命中输出：关键词、命中数、资料类型、是否可引用。
3. 对外部资料输出：一手来源、论文、综述、实践项目。
4. 最终报告必须有 Source Role Map。

验证：

输入一个同时包含本地资料和 Web 搜索要求的研究任务。

检查最终输出是否：

- 先说明本地发现。
- 再说明英文资料。
- 区分实践线索和论文主引用。
- 不泄露私人聊天细节。

这就是一次 Harness 自进化：不是让模型“记住要认真”，而是改变研究流程和输出标准。

自进化案例：Coding Agent Harness

初始失败：

Agent 修改代码后说测试通过，但其实没有运行测试。

失败层级：

Workflow：缺少验证 gate。

Observability：没有保存测试命令和输出。

Permissions：Agent 可以直接声称完成。

Harness 改进：

1. Stop gate 检查是否有 test evidence。
2. Final response 必须列出实际运行的命令。

3. 没有测试证据时不能说“通过”。
4. 新增 regression eval: 检查报告是否包含未运行测试的虚假声明。

更强的版本可以加 hook:

如果检测到代码 diff 且没有 test evidence,
就阻止 completion,
要求 Agent 运行测试或明确说明未验证原因。

这类改进比多写一句“请务必测试”可靠，因为它把行为约束放进了系统。

自进化案例：飞书文档导出 Harness

初始失败：

创建飞书文档时 API 权限不足，流程执行到中途才失败。

失败层级：

Tool: 缺少 permission preflight。
Workflow: 缺少 fallback。
Observability: 缺少 export metadata。

Harness 改进：

1. 开始前检查 docx:create 权限。
2. 如果权限缺失，直接输出授权 URL。
3. 同时生成本地 DOCX fallback。
4. 保存导出尝试记录。

验证：

在权限不足场景下，流程不应该中途失败；
应该清楚给出缺失权限和本地替代产物。

这类工具层进化非常实用，因为它能减少同类失败的恢复成本。

自进化设计题

如果你要给自己的 Agent workflow 加自进化，
可以按下面的问题设计：

1. 最近三次用户纠正是什么？
2. 最近三次工具失败是什么？
3. 最近三次“说完成但没完成”的情况是什么？
4. 哪些失败可以转化为 `eval`？
5. 哪些失败应该转化为 `memory`？
6. 哪些失败应该转化为 `skill`？
7. 哪些失败应该通过 `tool wrapper` 解决？
8. 哪些失败不应该自动化，只应该提醒人工审查？
9. 哪些旧规则已经失效？
10. 下次同类任务如何验证系统确实变好了？

把这十个问题回答清楚，你就有了一套自进化路线图。

本章结论

Harness 的高级形态不是“写一套完美规则”，
而是“建立一个会从运行中变好的系统”。

真正的自进化包含四句话：

每次失败都要留下证据。

每个证据都要定位层级。

每个高价值失败都要产生改进候选。

每个改进都要经过验证再进入正式 Harness。

能做到这四点，Harness 就开始有生命力。

7. Harness 的九层结构：九个可进化面

本章主线：九层结构不是静态清单，而是九个可进化面。每一层都要回答三件事：什么信号说明它该变、怎么变、变完如何验证。

本章学习目标

你要把 Harness 看成一个分层系统。分层的好处是，当 Agent 失败时，你可以定位到底是哪一层有问题，而不是笼统地说“AI 不行”。

九层之间的依赖关系

九层不是并列清单，而是有依赖关系：

- 任务规格决定需要什么上下文
- 上下文决定 Agent 如何理解任务
- 工具决定 Agent 能采取什么行动
- 权限决定行动边界
- 记忆决定跨任务复用
- 技能决定过程复用
- 工作流决定顺序和角色
- 验证决定是否完成
- 可观测性决定如何复盘和改进

如果任务规格错了，后面全都会偏。

如果工具不可用，再好的计划也没法执行。

如果验证缺失，系统不知道自己错了。

如果可观测性缺失，系统不知道怎么改。



层	诊断问题
任务规格	是否要求优先找一手来源？
上下文	是否给了已有 URL 或关键词？
工具	是否有联网搜索和网页展开工具？
权限	是否允许访问网络？
工作流	是否有 source discovery 阶段？
验证	是否检查 top sources 覆盖？
可观测性	是否记录了搜索过程？

这样你会发现，问题可能不是模型，而是没有设计 source coverage eval。

学习建议

后面每学一层，都问自己：

- 这一层解决什么失败模式？
- 如果缺这一层，会发生什么？
- 我现在的 Agent 工作流有没有这一层？
- 这一层最小可用版本是什么？

为了学习和设计方便，可以把 Harness 分成九层：

9. 可观测性和反馈回路
 8. 验证和评测
 7. 工作流和协议
 6. 技能
 5. 记忆

4. 执行环境和权限
3. 工具
2. 上下文
1. 任务规格

这九层不是必须全部一次性做完。一个最小 Harness 可能只有任务规格、工具、验证和日志。一个成熟 Harness 才会有记忆、技能、角色分工、长任务恢复和持续 eval。

学习时要避免两个误区。

第一个误区是“上来就做全套”。这会让 Harness 变成沉重的框架，最后没人愿意用。

第二个误区是“只做 prompt”。这会让系统没有反馈，无法改进。

好的做法是从真实失败出发。Agent 哪一步最容易错，你就优先补哪一层。

8. 第一层：任务规格

自进化视角：任务规格的进化信号，通常来自用户纠正、验收失败和“做了很多但不是用户要的”。规格不是写完不动，而是从误解中长出来。

本章学习目标

任务规格是 Harness 的地基。你要学会把一个模糊请求改写成 Agent 可以执行、系统可以检查、人可以验收的规格。

读完本章，你应该能写出：

- 目标清楚的 Goal。
- 可执行的 Scope。
- 明确的 Non-goals。
- 可检查的 Acceptance Criteria。
- 风险和人工确认点。

为什么 Agent 特别需要任务规格

人类同事在任务模糊时，会根据组织背景、过往经验、风险意识主动问问题。Agent 也会问问题，但它更容易“自动补全”。

例如你说：

帮我整理一下这个项目。

Agent 可能理解成：

- 重构目录。
- 改 README。
- 删除旧文件。
- 格式化代码。
- 做一份报告。
- 归档数据。

这些都可能合理，但不一定是你要的。

任务规格的作用就是减少自动补全空间。

任务规格的三种粒度

粗粒度规格

适合探索性任务：

目标：调研 Harness Engineering 的核心概念。

输出：一份中文学习笔记。

限制：不要改代码，不要暴露私有数据。

粗粒度规格允许 Agent 自主探索，但风险是产出可能偏离预期。

中粒度规格

适合大部分工作：

目标：写一份 Harness Engineering 入门小册子。

读者：有工程背景但不熟悉 Agent 系统的人。

结构：概念、组件、案例、模板、练习。

验收：读者看完能设计一个最小 Harness。

限制：不要把资料收集工具作为主题，只作为背景。

中粒度规格最实用。它给方向，也留空间。

细粒度规格

适合高风险或重复任务：

目标：生成客户导入 CSV。

输入：指定 Excel 文件。

字段：phone、name、source、owner。

规则：phone 必须去空格，重复手机号只保留第一条。

验收：输出行数等于唯一手机号数；生成重复列表；不修改原文件。

细粒度规格降低自由度，适合数据、生产、发送、发布等任务。

Acceptance Criteria 要能检查

差的验收：

报告要完整、深入、有价值。

好的验收：

报告至少包含：

- Harness 定义。
- 与 Prompt/Context/Eval 的区别。
- 9 个核心组件。
- 3 个具体场景。
- 1 个从零设计流程。
- 模板和练习。

“完整、深入、有价值”可以作为方向，但不能作为唯一验收。

Non-goals 很重要

很多事故不是因为 Agent 没做目标，而是因为它做了目标之外的事。

Non-goals 示例：

Non-goals

- 不实现代码。
- 不改数据库 schema。
- 不提交 git。
- 不联系外部用户。
- 不输出私有聊天原文。
- 不评价无关产品。

Non-goals 是 Harness 的护栏。

任务规格检查问题

写完 Task Spec 后，问：

- 目标是否只有一个主目标？

- 范围是否明确？
- 有没有写不做什么？
- 验收标准能否被检查？
- 有没有说明输出格式？
- 有没有高风险动作需要批准？
- Agent 如果照着做，是否还需要猜很多？

7.1 为什么任务规格重要

Agent 不擅长读心。模糊目标会被它自动补全，而补全出来的东西可能不是你想要的。

例如：

帮我优化这个系统。

这句话对 Agent 来说太宽。它可能优化性能，可能重构代码，可能改 UI，可能写文档，可能做一堆你没要求的事。

更好的规格是：

目标：降低搜索接口 P95 延迟。

范围：只允许修改 `search service` 和相关测试。

不做：不改数据库 `schema`，不改 API 返回结构。

验收：现有测试通过，新增 `benchmark` 显示 P95 降低至少 30%。

输出：说明改动、风险、验证命令。

任务规格的作用是减少 Agent 自由发挥的空间，把“我要什么”转成“怎么判断完成”。

7.2 好任务规格的五要素

一个好规格至少包括：

1. 目标：要达成什么。
2. 范围：可以动哪里，不能动哪里。
3. 约束：必须遵守什么规则。
4. 验收：怎么判断完成。
5. 输出：最终要交付什么。

模板：

```
## Task Spec

### Goal
...

### Scope
...

### Non-goals
...

### Constraints
...

### Acceptance Criteria
...

### Required Output
...
```

7.3 规格不是越长越好

规格的目标不是把所有可能性写死，而是把关键边界写清楚。

差的规格：

请确保所有代码优雅、健壮、高性能、可维护、符合最佳实践。

这类词没有判断标准。Agent 看了也只能猜。

好的规格：

不要新增依赖。

不要修改 `public API`。

新增失败用例覆盖空列表输入。

运行 `go test ./...`。

最终说明是否修改了数据库访问路径。

越能被检查的规格，越适合作为 Harness 的输入。

7.4 学习练习

找一个你最近交给 Agent 的任务，把它改写成 Task Spec。重点写清：

- 什么算完成。
 - 哪些事不允许做。
 - 哪些结果必须用命令验证。
-

9. 第二层：上下文

自进化视角：上下文的进化目标不是越多越好，而是让 source of truth 更清楚、让过期材料退出、让高价值经验进入 playbook。这里最容易出现 context collapse。

本章学习目标

上下文层解决“Agent 应该知道什么”。本章要让你从“把资料都塞给模型”转向“按任务设计 Context Pack”。

上下文污染

上下文不只是帮助，也可能污染。

常见污染包括：

- 过期文档。
- 错误历史结论。
- 无关文件。
- 低质量网页。
- 用户随口猜测。
- 旧版本 API 文档。

- 其他项目的规则。

Agent 很难天然判断哪些上下文可信。如果你把旧规则和新规则一起塞进去，它可能选择旧规则。

Source of Truth

每个 Harness 都应该有 Source of Truth。

例如代码项目：

Source of Truth:

- 当前源码。
- 当前测试。
- AGENTS.md。
- README 中的运行方式。

Not source of truth:

- 旧聊天记录。
- 过期 issue 评论。
- 外部博客里的通用建议。

例如研究任务：

Source of Truth:

- 官方文档。
- 论文。
- 源码。
- 一手采访。

Supporting context:

- 社区文章。
- 本地讨论。
- 视频和社交媒体。

Source of Truth 的目的是让 Agent 在冲突时知道优先相信谁。

上下文的生命周期

上下文不是一次性加载，而是有生命周期：

发现 -> 筛选 -> 压缩 -> 使用 -> 更新 -> 废弃

很多 Harness 失败，是因为只做了发现，没有做废弃。旧上下文一直堆在系统里，最后变成负担。

上下文压缩

长文档不应该总是全文进入上下文。可以压缩成：

- 摘要。
- 关键约束。
- API 表。
- 决策记录。
- 示例片段。
- 文件路径和行号。

但压缩有风险：摘要可能遗漏细节。所以关键事实要保留来源。

Context Pack 设计例子

Research Harness 的 Context Pack:

Context Pack: Concept Research

Required:

- Research question.
- Local discussion summary.
- URL frequency table.
- Expanded primary sources.
- Source credibility labels.

Optional:

- Community commentary.
- Videos and social signals.
- Related GitHub projects.

Do not include:

- Private personal IDs.
- Full chat logs.
- Private attachment URLs.
- Irrelevant URLs only containing the keyword accidentally.

Coding Harness 的 Context Pack:

Context Pack: Bug Fix

Required:

- Bug description.
- Reproduction steps.
- Failing test or error log.
- Related source files.
- Test command.

Optional:

- Similar previous bugs.
- Architecture notes.

- Recent commits.

Do not include:

- Entire repo dump.
- Generated files.
- Secrets.

判断上下文是否足够

不是看上下文多不多，而是看 Agent 能否回答：

- 我现在要完成什么？
- 哪些事实最可靠？
- 哪些文件或来源最相关？
- 哪些信息可能过期？
- 如果信息冲突，我该相信谁？

8.1 Context 不是越多越好

很多人以为给 Agent 塞更多上下文，结果就会更好。实际并不总是这样。

上下文有三个问题：

1. 它占 token。
2. 它会稀释注意力。
3. 它可能包含过期或错误信息。

好的 Context Engineering 不是“全塞进去”，而是让 Agent 在正确时间看到正确信息。

8.2 上下文的类型

常见上下文可以分为：

类型	示例	风险
项目地图	README、目录结构、AGENTS.md	太长会淹没重点
任务上下文	用户目标、当前问题、日志	不完整会误导
代码上下文	相关文件、接口、测试	选错文件会走偏
历史上下文	过去决策、已知坑	可能过期
运行上下文	命令输出、错误日志、截图	可能是偶发现象
外部知识	文档、网页、论文	可能不适用于本项目

8.3 上下文的三条原则

第一，先地图，后细节。

Agent 初到一个项目，需要先知道边界和入口，而不是立刻吞下所有文件。

第二，按需读取。

让 Agent 根据任务逐步打开相关文件，比一次性喂大包更可靠。

第三，保留来源。

上下文必须能追溯来源。尤其是研究任务，不能只有“有人说”，要知道谁说、什么时候说、原文在哪里。

8.4 Context Pack

可以把某类任务需要的上下文打包成 Context Pack。

示例：

```
## Context Pack: Bug Fix
```

Required:

- Task Spec
- Error log
- Related source files
- Existing tests
- How to run tests

Optional:

- Recent commit history
- Similar past bugs
- Architecture notes

Do not include:

- Unrelated files
- Large generated assets
- Secrets

8.5 对长任务的特殊处理

长任务不能依赖对话上下文一直存在。要把关键状态写进 handoff artifact：

```
## Handoff
```

```
### Goal
```

```
...
```

```
### Current State
```

```
...
```

```
### Decisions Made
```

```
...
```

```
### Files Changed
```

```
...
```

```
### Tests Run
```

```
...
```

```
### Open Issues
```

```
...
```

```
### Next Step
```

```
...
```

没有 handoff，长任务很容易在 context reset 后丢失方向。

10. 第三层：工具

自进化视角：工具层最适合把重复提醒变成机械约束。凡是同类工具错误重复出现，就应该考虑 wrapper、preflight、redaction、schema 或权限限制。

本章学习目标

工具层解决“Agent 能做什么”。本章要让你学会设计 Agent-friendly tools，而不是简单把人类工具丢给 Agent。

人类工具和 Agent 工具的区别

很多工具是给人类设计的：

- 输出花哨。
- 需要交互。
- 错误信息隐蔽。
- 默认分页。
- 依赖视觉判断。
- 状态保存在 UI 里。

Agent 更适合：

- 结构化输入。
- JSON/JSONL 输出。
- 可重复命令。
- 明确错误码。
- 可限制范围。
- 可 dry-run。

Tool Wrapper

很多时候，不需要改原工具，只需要加 wrapper。

原命令：

```
some-tool list
```

可能输出一大堆给人看的彩色文本。

Wrapper 可以变成：

```
some-tool-json list --limit 100 --format json
```

给 Agent 的输出更稳定。

工具设计的六个问题

设计工具前问：

1. Agent 为什么需要这个工具？

- 2. 输入最小参数是什么？
- 3. 输出是否结构化？
- 4. 失败时是否有可读错误？
- 5. 是否有只读或 dry-run 模式？
- 6. 是否能限制作用范围？

工具的危險等級

工具类型	风险	策略
读取文件	低到中	默认允许，但保护秘密
搜索网页	中	需要来源记录
写工作区文件	中	限制目录，记录 diff
安装依赖	中到高	需要批准
发送消息	高	需要明确用户确认
删除数据	高	默认禁止或强批准
改生产系统	极高	专门流程和审计

工具输出的“可观察性”

Agent 调用工具后，不只是要拿结果，还要知道：

- 工具是否成功。
- 用了什么参数。
- 影响了什么对象。
- 结果有多少条。

- 是否被截断。
- 是否有警告。

所以工具输出最好包含 meta:

```
{  
  "meta": {  
    "status": "ok",  
    "count": 120,  
    "truncated": false,  
    "source": "local_index"  
  },  
  "items": []  
}
```

工具不是越多越好

工具太多会增加选择成本。Agent 可能用错工具。

好的 Harness 会把工具分层：

- 核心工具：高频、稳定、默认可用。
- 专项工具：特定任务触发。
- 高风险工具：需要批准。
- 废弃工具：明确不要使用。

9.1 工具决定 Agent 能力边界

Agent 的能力不是模型单独决定的。它能调用什么工具，会极大影响它能做什么。

一个只能聊天的模型，很难完成真实工程任务。

一个能读文件、改文件、跑测试、查文档、开浏览器的 Agent，才接近真实生产力工具。

但工具越强，风险越大。

强工具 + 弱权限 = 风险

弱工具 + 高期望 = 低效

强工具 + 好 Harness = 生产力

9.2 好工具的特征

一个适合 Agent 使用的工具应该：

- 输入结构清晰。
- 输出稳定。
- 错误信息可读。
- 支持 dry-run 或只读模式。
- 支持小范围操作。
- 能被日志记录。
- 不默认做破坏性动作。

坏工具通常是：

- 输出全靠人眼解析。
- 失败时没有错误码。
- 一次执行影响范围过大。
- 需要复杂交互。

- 默认修改大量状态。

9.3 工具说明要写给 Agent

人类知道 `git reset --hard` 危险，Agent 不一定天然知道。所以工具说明里要写清：

```
## Tool Policy
```

```
Allowed:
```

- read files
- run tests
- inspect git diff

```
Requires approval:
```

- install dependencies
- network calls
- write outside workspace

```
Forbidden unless explicitly requested:
```

- git reset --hard
- deleting user data
- printing secrets

9.4 工具输出要可压缩

Agent 工具输出很容易变成长日志。长日志不仅贵，还会干扰判断。

建议：

- 默认输出摘要。
- 失败时输出关键错误。
- 支持 `--json`。

- 支持分页和 limit。
- 大结果保存为 artifact，只把路径和摘要给 Agent。

9.5 学习练习

选一个你常用工具，问自己：

- 它适合 Agent 调用吗？
 - 它的输出是否稳定？
 - 它失败时是否容易诊断？
 - 是否需要加一个 wrapper？
-

11. 第四层：执行环境和权限

自进化视角：权限不能因为“提升效率”自动放宽。低风险动作可以自动化，高风险动作必须保留人工 gate；权限策略本身也需要版本和审计。

本章学习目标

权限层解决“Agent 能不能这么做”。本章要让你学会把安全限制设计成生产力，而不是阻碍。

权限设计的基本矛盾

Agent 权限太小，会不停卡住，需要人手动操作。

Agent 权限太大，可能误删数据、泄露秘密、发布错误内容。

好的权限设计，是让低风险动作自动化，高风险动作显式审批。

默认策略

可以采用这样的默认策略：

读操作默认允许

工作区写操作按任务允许

外部网络按任务允许

跨目录写入需要批准

发送、发布、删除、付费、生产变更必须批准

秘密输出默认禁止

Approval Gate

Approval Gate 不是简单弹窗，而是要说明：

- Agent 想做什么。
- 为什么需要做。
- 会影响什么范围。
- 如果不做有什么替代方案。
- 是否可以以后对同类命令默认允许。

坏的 approval:

Allow command?

好的 approval:

需要联网安装 Playwright 依赖，用于运行浏览器验证。会写入当前项目依赖缓存，不会修改业务文件。是否允许？

权限和日志绑定

每次高权限动作都要进入日志：

Escalated Action

Action: install browser dependency

Reason: needed for UI verification

Approved by: user

Scope: local workspace

Result: success

没有日志，审批无法复盘。

环境隔离

成熟 Harness 会区分：

- 本地开发环境。
- 临时 worktree。
- sandbox。
- staging。
- production。

Agent 默认应在低风险环境工作。生产环境动作必须有独立流程。

权限失败也是信号

如果 Agent 经常因为权限卡住，有两种可能：

1. 权限确实太窄。
2. 任务规格不该让 Agent 做这些动作。

不要机械放权，要先判断任务性质。

10.1 为什么权限是 Harness 的核心

Agent 会行动，所以必须有权限边界。

权限不是为了限制生产力，而是为了让 Agent 能安全行动。没有权限系统时，人会不敢放手；有合理权限时，人才能让 Agent 自动做更多事。

10.2 权限分级

可以把权限分成四级：

等级	说明	示例
Read-only	只能读取	搜索文件、读数据库、查看网页
Workspace-write	可写当前工作区	改代码、生成报告
Escalated	需要批准	安装依赖、访问网络、写系统目录
Forbidden	默认禁止	删除用户数据、泄露密钥、重置未授权变更

10.3 Sandbox

Sandbox 的作用是让 Agent 在有限空间里工作：

- 文件系统限制。
- 网络限制。
- 命令审批。

- 环境变量隔离。
- 临时目录隔离。

没有 sandbox 的 Agent 很难被信任。

有 sandbox 但没有审批机制，会让 Agent 很难完成真实任务。

好的 Harness 要在安全和效率之间找到平衡。

10.4 权限要和任务绑定

不同任务需要不同权限。

读代码解释问题：read-only

修改单元测试：workspace-write

安装依赖跑浏览器测试：需要审批

清理生产数据库：默认禁止

不要给所有任务同样权限。权限应该随任务规格动态决定。

12. 第五层：记忆

自进化视角：记忆是经验进入系统的入口，但也是污染入口。本书采用 candidate、active、scoped、archived 四态管理，而不是把所有反馈永久写入全局记忆。

本章学习目标

记忆层解决“Agent 如何不重复犯错”。本章要让你理解记忆不是聊天历史，而是可复用、可验证、会影响未来行为的知识。

三种记忆

事实记忆

稳定事实，例如：

- 项目测试命令。
- 配置文件路径。
- 某个业务口径。

偏好记忆

用户或团队偏好，例如：

- 报告要 Markdown。
- 先给结论再给细节。
- 不要输出隐私数据。

经验记忆

从失败中得到的经验，例如：

- 某命令在 sandbox 里会失败，需要审批。
- 某网站需要浏览器展开，普通 HTTP 抓不到正文。
- 某类数据必须先按业务口径拆 count。

记忆的质量问题

记忆也会坏。

坏记忆包括：

- 过期。
- 太泛。
- 没来源。
- 含秘密。
- 和当前任务无关。
- 把一次偶然现象当规律。

例如：

“这个项目很复杂。”

这没用。

更好的是：

“这个项目的 `help` 行为有明确契约：`bare command` 应显示 `help`，不应执行查询。”

记忆进入 Harness 的方式

记忆可以进入：

- AGENTS.md。
- README。
- skill 文档。
- runbook。
- decision record。
- local memory store。
- checklist。

关键是：记忆要能被 Agent 在正确时机读到。

什么时候不应该记忆

不要记：

- 临时情绪。
- 一次性命令输出。
- 可能含隐私的原文。

- 未验证猜测。
- 与未来行为无关的信息。

记忆不是越多越好。好的记忆是让未来行动更稳。

11.1 记忆解决什么问题

LLM 对话有上下文窗口，Agent 任务有 session 边界。没有外部记忆时，Agent 会反复遇到同样问题：

- 不记得项目惯例。
- 不记得用户偏好。
- 不记得之前踩过的坑。
- 不记得某个工具怎么用。
- 不记得某个设计为什么这样定。

Memory Layer 的作用是把这些信息外部化。

11.2 记忆的类型

类型	示例
用户偏好	不要暴露个人 ID，喜欢 Markdown 交付
项目规则	测试命令、目录约定、代码风格
历史决策	为什么不用某个库，为什么保留某个接口
失败模式	某命令会卡住，某 API 有分页坑
技能经验	如何展开微信文章，如何跑浏览器测试

11.3 好记忆的标准

好记忆应该：

- 简短。
- 可验证。
- 有来源。
- 不含秘密。
- 能被更新。
- 不把临时事实当长期规则。

坏记忆：

用户喜欢高质量。

这没有执行价值。

好记忆：

在本地数据工具相关任务中，不要输出完整 DB key、个人账号或私有附件链接。

这能直接约束行为。

11.4 记忆不是垃圾桶

不要把所有对话都塞进 memory。太多记忆会污染上下文。

记忆应该只保存会影响未来行为的信息：

- 稳定偏好。
 - 项目契约。
 - 重复失败。
 - 可复用流程。
 - 关键决策。
-

13. 第六层：技能

自进化视角：Skill 是被验证过的流程资产。一个经验只有在反复出现、需要步骤化、并能验证效果时，才应该从 memory 升级为 skill。

本章学习目标

技能层解决“可复用过程”。本章要让你知道，Skill 不是一个漂亮名称，而是一套能重复执行的流程。

Skill 和 Rule 的区别

Rule 告诉 Agent 应该遵守什么。

Skill 告诉 Agent 遇到某类任务该怎么做。

Rule 示例：

不要输出私人 ID。

Skill 示例：

当需要从本地聊天生成公开报告时：

1. 先抽取主题和 URL。
2. 标记私有链接。

3. 只使用公开来源支撑事实。
4. 输出前运行隐私检查。

Rule 是约束，Skill 是流程。

Skill 的触发条件

一个 Skill 必须写清什么时候用。不然 Agent 可能用错。

差：

```
Use this skill for research.
```

好：

```
Use this skill when the task requires turning  
local private discussion and public  
URLs into a public-facing research  
report.
```

Skill 的输入输出

Skill 必须明确输入输出。

例如：

Inputs:

- topic
- local hit file
- URL list
- source expansion results

Outputs:

- evidence ledger

- source summary
- public report
- failed URL list

没有输入输出，Skill 就无法组合。

Skill 的测试

成熟 Skill 应该有测试或样例：

- 给一组 URL，是否能正确 dedupe。
- 给一段含私人 ID 的文本，是否能脱敏。
- 给一个失效链接，是否能标记 failed。
- 给一个社区观点，是否不会当一手事实。

Skill 不只是文档，也可以被 eval。

12.1 Skill 是可复用过程

Skill 不是“我会做某事”的一句话。它应该是可复用的操作流程。

例如，“做研究”太泛。一个好的 research skill 应该写清：

- 什么时候使用。
- 输入是什么。
- 搜索策略是什么。
- 如何判断来源可信。
- 如何记录证据。

- 如何输出报告。
- 如何处理失败。

12.2 Skill 的结构

一个实用 Skill 可以这样写：

```
# Skill: URL Source Expansion
```

```
## When to use
```

```
When a local discussion contains URLs that need  
to be expanded and summarized.
```

```
## Inputs
```

- URL list
- Topic
- Max sources

```
## Steps
```

1. Deduplicate URLs.
2. Classify domains.
3. Expand public URLs.
4. Mark private or expired URLs.
5. Summarize each source.
6. Write evidence records.

```
## Output
```

- sources.jsonl
- failed_urls.csv
- summary.md

```
## Safety
```

- Do not expose private attachment URLs.
- Do not copy long copyrighted passages.

12.3 Skill 和脚本的关系

最好的 Skill 通常包含脚本。自然语言说明容易被误解，脚本能保证稳定行为。

Skill = 说明 + 步骤 + 输入输出 + 脚本 + 验证

如果一个 Skill 每次执行都靠人重新解释，那它还不是成熟 Skill。

12.4 Skill 的演进

Skill 应该从失败中长出来：

1. Agent 做错一次。
 2. 找到可复用的修正步骤。
 3. 把步骤写成 Skill。
 4. 下次遇到同类任务先调用 Skill。
 5. 如果仍失败，更新 Skill。
-

14. 第七层： workflow 和协议

自进化视角： workflow 决定失败在哪个阶段被发现。好的 workflow 不只是安排顺序，还会定义 checkpoint、rollback、handoff 和 promotion gate。

本章学习目标

workflow 层解决“先做什么，后做什么”。协议层解决“遇到情况怎么办”。本章要让你能把复杂任务拆成稳定流程。

为什么流程顺序重要

Agent 经常犯的错误是跳步。

研究任务跳过 source discovery，直接写报告。

代码任务跳过 read context，直接改文件。

数据任务跳过 schema inspection，直接清洗。

浏览器任务跳过登录检查，直接点击按钮。

Workflow 就是防跳步结构。

工作流的粒度

工作流太粗：

研究 -> 写报告

没有约束。

工作流太细：

打开第一个链接 -> 读第一段 -> 摘要第一段 -> ...

太死板。

合适的粒度是阶段级：

定义问题 -> 发现来源 -> 展开来源 -> 建证据表 -> 综合判断 -> 审查报告

每个阶段有输入输出和验收。

协议处理异常

Workflow 处理正常路径，Protocol 处理异常路径。

例如 URL 展开协议：

If URL is public and accessible:

expand and summarize.

If URL requires login:

mark login_required and do not use as public evidence.

If URL is private attachment:
mark private_artifact and do not expose.

If URL is duplicated:
merge mention count.

If URL is off topic:
mark off_topic with reason.

没有协议，Agent 遇到异常就会自由发挥。

workflow 中的检查点

复杂任务不应该等到最后才检查。

可以设置检查点：

- 资料发现后：来源够不够。
- 计划生成后：范围是否正确。
- 初稿完成后：结构是否符合要求。
- 最终前：验证和隐私检查。

检查点越早，返工成本越低。

13.1 Workflow 解决任务顺序

很多任务不是一步完成，而是有阶段。

以研究报告为例：

定义问题 -> 搜索本地资料 -> 抽 URL -> 展开来源 -> 建证据表 -> 写大纲 -> 写报告 -> 审查

如果没有 workflow，Agent 可能跳过关键步骤，直接生成一份看似完整但证据不足的报告。

13.2 Protocol 解决交互规则

Protocol 是 workflow 中的约定。例如：

- 什么时候必须问用户。
- 什么时候可以自主决定。
- 什么时候必须停止。
- 什么时候要保存 artifact。
- 什么时候要升级权限。
- 什么时候需要 reviewer。

例子：

Research Protocol

1. If source count is below 5, do not produce final conclusions.
2. If a URL requires login, mark it as private/login-required.
3. If a source is community commentary, do not treat it as primary evidence.
4. If a claim appears in only one weak source, mark confidence as low.
5. Before final report, run privacy check.

13.3 Handoff Protocol

长任务中，handoff 是核心协议。

一个好的 handoff 要让下一个 Agent 或下一轮 session 不需要重新猜：

```
## Handoff

### Objective
...

### Completed
...

### Current Evidence
...

### Decisions
...

### Risks
...

### Next Actions
...
```

Handoff 不是总结给人看的漂亮话，而是给下一步执行用的状态包。

15. 第八层：验证和评测

自进化视角：Eval 是进化闸门。没有 eval，改进候选无法判断是否真的变好；没有 regression eval，系统会在解决旧问题时制造新问题。

本章学习目标

验证层解决“结果是不是真的对”。本章要让你把完成感从主观判断变成证据。

Verification 和 Evaluation 的区别

Verification 更偏确定性：

测试是否通过
文件是否存在
URL 是否可访问
JSON 是否符合 schema
count 是否一致

Evaluation 更偏质量判断：

报告是否深入
建议是否可落地
UI 是否好用
回答是否覆盖关键反例

两者都需要，但顺序应该是：

先 Verification
再 Evaluation

如果文件都没生成，就不要讨论写得好不好。
如果测试都没跑，就不要讨论代码风格。
如果来源都没展开，就不要讨论报告深度。

验证点要贴近失败模式

不要为了验证而验证。每个验证点都应该对应一个真实风险。

风险	验证
Agent 没跑测试	检查测试命令输出
报告漏来源	检查 top URL coverage
数据行数错	count reconciliation
输出泄露隐私	privacy scanner
UI 空白	screenshot check
长任务丢状态	handoff completeness check

LLM Judge 的正确位置

LLM Judge 适合做：

- 结构完整性审查。
- 逻辑连贯性审查。
- 反例提醒。

- 读者视角反馈。
- 主观质量评分。

不适合替代：

- 编译。
- 单元测试。
- HTTP 状态检查。
- schema 校验。
- 权限审计。
- 秘密扫描。

Eval 的版本化

Harness 会变，eval 也要版本化。

```
eval/research_report_v1  
eval/research_report_v2  
eval/coding_bugfix_v1
```

否则你无法比较不同 Harness 版本是否真的变好。

14.1 没有验证就没有可靠 Agent

Agent 最危险的问题之一是假完成。它可能非常自信地说任务完成了，但实际上：

- 测试没跑。
- 跑错目录。

- 忽略失败输出。
- 只验证 happy path。
- 报告里引用了失效链接。
- 代码能编译但逻辑错。

所以 Harness 必须有 Verification Layer。

14.2 验证的三种类型

第一，确定性验证。

测试、lint、类型检查、schema 校验、URL 状态码、文件存在性、count 对齐

确定性验证最可靠，应该优先使用。

第二，半结构化验证。

rubric 打分、review checklist、覆盖率检查、引用检查

这类验证需要一定判断，但可以结构化。

第三，语义验证。

LLM judge、专家 review、用户验收

语义验证适合检查写作质量、产品体验、推理完整性等主观问题。

14.3 Eval 不是测试的替代品

Eval 很重要，但不要把所有东西都交给 LLM judge。

如果可以用程序检查，就不要用语义判断：

问题	更好的检查方式
文件是否生成	文件系统检查
URL 是否可访问	HTTP 状态检查
JSON 是否符合 schema	schema validator
代码是否编译	编译命令
测试是否通过	test runner
是否泄露 ID	正则和规则扫描

LLM judge 应该处理程序难以判断的部分：

- 报告是否逻辑清楚。
- 是否遗漏重要反例。
- 结论是否过度推断。
- 建议是否可落地。

14.4 Golden Tasks

要评估 Harness 是否进步，需要固定任务集。

例如 Research Harness 可以有这些 golden tasks：

- 从本地聊天研究一个新概念。
- 从一组微信群链接生成竞品分析。
- 从一批 GitHub repo 总结技术路线。
- 从多个观点冲突的来源形成判断。

每个任务都保存：

- 输入。
- 期望覆盖的来源。
- 必须避免的错误。
- 最低报告质量。

没有 golden tasks，Harness 优化会变成主观感觉。

16. 第九层：可观测性和反馈回路

自进化视角：可观测性提供进化原料。没有 trace、log、artifact、verification result，就没有 failure attribution，也就没有可靠自进化。

本章学习目标

可观测性层解决“我们怎么知道 Agent 做了什么”。反馈回路解决“我们怎么让下次更好”。这是 Harness 从流程走向工程系统的关键。

Trace、Log、Artifact

可以区分三类记录。

Trace 是过程细节：

- 调用了哪个工具。
- 参数是什么。
- 返回了什么。
- 用了多长时间。

Log 是人可读记录：

- 这次任务做了什么。
- 遇到什么问题。
- 怎么解决。
- 还有什么风险。

Artifact 是产物：

- 报告。
- CSV。
- 代码 diff。
- 截图。
- evidence ledger。

三者都重要。Trace 适合调试，Log 适合复盘，Artifact 适合交付。

反馈不等于总结

很多 Agent 会在最后写：

本次任务顺利完成。

这不是反馈。

真正的反馈应该能改变系统：

Failure:

We missed two high-frequency URLs because URL extraction regex stopped at Chinese punctuation.

Harness Change:

Update URL extraction rule and add regression sample.

Feedback Loop 的四步

发现问题 -> 定位原因 -> 改 Harness -> 用同类任务验证

如果只发现问题但不改 Harness，下次还会错。

如果改了 Harness 但不验证，无法知道是否有效。

应该进入反馈的内容

适合进入反馈：

- 高频失败。
- 高风险失败。
- 明确可修复的流程缺陷。
- 用户反复纠正的偏好。
- 工具输出格式问题。

不适合进入反馈：

- 一次偶发网络失败。
- 用户临时改变主意。
- 无法复现的错误。
- 和未来任务无关的细节。

15.1 为什么日志重要

没有日志，就无法回答：

- Agent 为什么做这个决定？
- 它看过哪些文件？
- 它漏掉了什么来源？
- 哪个工具失败了？
- 失败是否可复现？
- 下次应该改什么？

Harness 的目标不是让 Agent 永不犯错，而是让错误能被发现、定位和转化为改进。

15.2 需要记录什么

最小 run log：

```
## Run Log

### Task
...

### Inputs
...

### Tools Used
...

### Key Observations
...

### Failures
...
```

```
### Verification
```

```
...
```

```
### Final Output
```

```
...
```

```
### Learnings
```

```
...
```

对代码任务，还要记录：

- 修改文件。
- 测试命令。
- 失败日志。
- 未验证项。

对研究任务，还要记录：

- 查询词。
- 命中数。
- URL 数。
- 展开成功/失败。
- 来源分类。
- 证据不足的结论。

15.3 反馈如何进入下一轮

反馈不应该只停留在“下次注意”。它应该进入系统：

失败	改进形式
忘记跑测试	completion checklist
引用失效 URL	URL validation eval
泄露个人 ID	privacy scanner
总结太空	report rubric
工具输出太长	tool output summarizer
重复读错文件	project map

这就是 Harness 的飞轮：



17. Human-in-the-loop: 人应该在哪里介入

自进化视角：人类不是流程里的监工，而是高风险改动、模糊价值判断和 promotion decision 的审查者。

本章学习目标

Human-in-the-loop 不是“人一直盯着 Agent”。本章要让你学会设计介入点，让人类只在高价值、高风险、高判断密度的位置出现。

人类介入的三种类型

1. Goal Steering

人定义目标和取舍。

例子：

- 这个报告是给新手看，还是给专家看？
- 这个功能优先稳定，还是优先速度？
- 这个数据口径以财务为准，还是以业务运营为准？

这些问题通常不能完全交给 Agent。

2. Risk Approval

人批准高风险动作。

例子：

- 发送消息。
- 删除数据。
- 发布版本。
- 修改生产配置。
- 支付费用。
- 暴露或处理敏感信息。

3. Quality Judgment

人判断主观质量和业务语境。

例子：

- 报告是否有洞察。
- UI 是否符合品牌。
- 回复客户是否合适。
- 方案是否符合组织现实。

不应该让人做什么

如果 Harness 设计得好，人不应该反复做这些事：

- 提醒 Agent 跑测试。
- 提醒 Agent 不要泄露隐私。

- 手动检查文件是否生成。
- 每次解释同一个项目规则。
- 每次纠正同一个输出格式。

这些应该进入规则、工具或 eval。

介入点设计模板

```
## Human Intervention Points
```

```
Agent may proceed automatically:
```

```
- ...
```

```
Agent must ask before:
```

```
- ...
```

```
Agent must stop if:
```

```
- ...
```

```
Human reviews:
```

```
- ...
```

好的介入设计

好的设计不是“人少参与”，而是“人在正确位置参与”。

低风险重复动作 -> Agent 自动化

中风险判断动作 -> Agent 提建议，人确认

高风险外部动作 -> 人明确批准

高价值战略判断 -> 人主导

16.1 人不是每一步都要盯着

如果人每一步都要确认，Agent 就退化成慢速助手。

如果人完全不介入，风险会失控。

Human-in-the-loop 的关键是确定介入点。

16.2 低风险任务让 Agent 自主完成

例如：

- 读取公开文档。
- 生成摘要。
- 运行只读查询。
- 修改临时文件。
- 生成草稿。

这些任务可以减少确认。

16.3 高风险任务必须人工批准

例如：

- 删除数据。
- 修改生产配置。
- 发布版本。
- 发送外部消息。
- 提交代码。
- 暴露私人数据。

- 使用付费或高成本资源。

16.4 人类最重要的角色

人类不应该只做“结果验收员”。更重要的是：

- 定义目标。
- 判断取舍。
- 审查高风险行为。
- 发现反复失败模式。
- 决定哪些经验要沉淀。

Agent 执行，Harness 约束，人类 steering。

18. 成本、缓存和速度

自进化视角：成本、缓存和速度会影响系统能否长期进化。上下文无限膨胀、重复展开资料、重复跑昂贵工具，都会让 Harness 变得不可持续。

本章学习目标

成本不是财务部门才关心的事。Agent Harness 的成本包括 token、时间、工具调用、人工注意力和失败返工。本章要让你把成本当成架构约束。

五类成本

成本	说明	例子
Token 成本	模型输入输出	大上下文、多轮对话
时间成本	等待执行	浏览器、测试、网络
工具成本	外部 API 或资源	搜索、云服务、付费模型
人工成本	人类审查和纠正	反复确认、手动复盘
返工成本	错误导致重做	错误报告、坏代码、数据错口径

很多系统只算 token，忽略人工和返工，这是不完整的。

缓存友好的上下文设计

Prompt caching 要求上下文前缀尽量稳定。
Harness 可以这样设计：

- 稳定规则放前面。
- 任务变量放后面。
- 工具说明顺序固定。
- 大型参考文档用引用和摘要。
- 频繁变化的日志不要塞进稳定前缀。

成本和准确性的取舍

不是所有任务都需要最强模型和最长上下文。

低风险格式转换：小模型 + 程序校验

代码修改：强模型 + 测试验证

战略判断：强模型 + 多来源 + 人类审查

批量重复任务：小模型/脚本 + 抽样 eval

优化成本的顺序

不要一开始就压 token。先保证闭环正确。

推荐顺序：

1. 先让流程正确。
2. 找出最贵步骤。

3. 缩短工具输出。
4. 增加缓存命中。
5. 把确定性任务交给脚本。
6. 把低价值语义判断移除。

注意力成本

人类注意力是最贵资源。一个 Harness 如果让人不断读长日志、确认小事、纠正重复错误，它就不合格。

真正好的 Harness 会把人的注意力集中到：

- 目标。
- 风险。
- 取舍。
- 最终质量。

17.1 成本是 Harness 的基础约束

很多 Agent 系统不是因为不能做，而是因为太慢、太贵、太不稳定。

成本包括：

- 输入 token。
- 输出 token。
- 工具调用时间。
- 网络请求。

- 浏览器自动化。
- 子 Agent 冷启动。
- 重复上下文。
- 人工 review 时间。

17.2 Prompt caching 为什么重要

长任务 Agent 往往有很高输入输出比。它每次要带大量上下文、工具说明、历史状态、代码片段。缓存命中率会直接影响成本和延迟。

Harness 设计要考虑：

- 稳定前缀。
- 工具说明顺序。
- 大文档拆分。
- 子任务边界。
- 不必要上下文裁剪。
- 复用中间 artifact。

17.3 快不等于好

Agent 太快也会放大错误。一个错误规格，如果被 Agent 高速执行，会产生更多错误结果。

所以速度要放在闭环之后：

先正确

再稳定

再快速

再规模化

19. Coding Agent Harness

自进化视角：Coding Harness 的优势是验证信号强。测试、lint、diff、CI、review 都可以成为进化门槛。

本章学习目标

Coding Agent 是最适合学习 Harness 的场景，因为它天然有文件、测试、版本控制、lint、CI 和 review。本章要让你能设计一个可靠的代码协作 Harness。

Coding Agent 的最小工作流

读任务

- > 读项目规则
- > 定位相关文件
- > 理解现有模式
- > 小范围修改
- > 运行相关测试
- > 检查 diff
- > 汇报验证和风险

任何跳过这些步骤的 Coding Agent 都容易出问题。

代码任务的 Scope Control

代码任务最重要的控制之一是范围。

Scope 应该包括：

- 可以修改的模块。
- 不可以修改的模块。
- 是否允许新增依赖。
- 是否允许改 public API。
- 是否允许改数据库 schema。
- 是否允许改测试快照。

示例：

Scope:

- Modify only ``pkg/search`` and related tests.
- Do not change API response shape.
- Do not add dependencies.
- Add regression test for empty query.

代码验证矩阵

不同改动需要不同验证。

改动类型	最小验证
文案/README	markdown lint 或人工检查
单函数 bugfix	相关单测
共享库修改	全量单测或模块测试
API 行为修改	单测 + 集成测试
性能优化	benchmark 或 profiling
UI 修改	浏览器截图和交互验证

Diff Review

Agent 完成后必须看 diff。Diff Review 问：

- 有没有不相关文件？
- 有没有格式化大面积改动？
- 有没有用户没要求的重构？
- 有没有秘密或本地路径？
- 测试是否真的覆盖改动？

Coding Harness 的升级路线

Level 1：

- AGENTS.md。
- 测试命令。
- diff review。

Level 2：

- 模块级规则。
- 常见错误 memory。
- 自动 lint/test。

Level 3:

- golden bugfix tasks。
- code review agent。
- CI feedback integration。
- benchmark suite。

18.1 Coding Agent 的典型问题

Coding Agent 常见失败:

- 没读项目规则就开始改。
- 修改范围过大。
- 没跑测试。
- 跑错测试。
- 修一个 bug 引入另一个 bug。
- 代码风格不一致。
- 没注意用户已有未提交改动。
- 不知道什么时候该问用户。

18.2 Coding Harness 的基本组成

最小 Coding Harness:

```
AGENTS.md / project rules
Task Spec
repo map
file search/read tools
edit tool
test commands
git diff inspection
completion checklist
```

更成熟的 Coding Harness:

```
architecture docs
module ownership
test matrix
lint/typecheck
CI integration
code review agent
failure memory
benchmark tasks
permission gates
```

18.3 Coding Harness Checklist

Before Coding

- [] Understand task goal.
- [] Identify scope.
- [] Read project rules.
- [] Inspect relevant files.
- [] Check existing tests.

During Coding

- [] Keep changes scoped.
- [] Preserve user changes.
- [] Follow local patterns.
- [] Add tests proportional to risk.

Before Completion

- [] Run relevant tests.
- [] Inspect diff.
- [] Note unresolved risks.
- [] Report verification result.

18.4 Coding Harness 的关键原则

不要让 Agent 靠“感觉”判断代码是否正确。让测试、类型系统、lint、编译、benchmark 和 review 共同工作。

20. Research Agent Harness

自进化视角：Research Harness 的进化重点是 source coverage、claim map、URL 展开、证据可信度和隐私边界。

本章学习目标

Research Harness 是最容易“看起来很强、实际上很虚”的场景。本章要让你学会用证据链约束研究输出。

Research Agent 的最大风险

Research Agent 最大风险不是写不出来，而是写得很像真的。

常见问题：

- 来源太少。
- 来源太弱。
- 只看标题。
- 没有打开 URL。
- 混淆事实和评论。
- 用社区观点替代一手证据。
- 结论没有置信度。

- 不说明没查到什么。

Research Harness 的关键产物

不要直接从资料跳到报告。中间应该有产物：

```
query log
URL table
source summaries
evidence ledger
claim map
contradiction list
open questions
report draft
review notes
```

这些中间产物让研究可复查。

来源可信度

来源可信度不是简单的“官方就一定对，社区就没用”。更准确的判断是：

来源	适合支撑	不适合支撑
官方文档	产品能力、设计意图	第三方效果评价
论文	理论框架、实验结果	产品落地体验
源码	实际实现	商业策略
GitHub issue	真实使用问题	普遍结论
社区文章	观点、传播、解释	强事实
本地讨论	线索、需求、问题意识	公开事实

Claim Map

研究报告里每个重要判断都应该能映射到证据。

Claim:

Harness is becoming a competitive layer outside model capability.

Evidence:

- OpenAI discusses agent-first engineering environment.
- LangChain shows benchmark gains from harness changes with same model.
- Cursor documents model-specific harness tuning.

Confidence:

High.

Caveat:

Benchmark gains do not automatically imply business task reliability.

研究报告的完成标准

Research Harness 应该要求报告回答：

- 我查了什么。
- 我没有查到什么。
- 哪些来源最重要。
- 哪些观点有冲突。
- 哪些结论高置信。
- 哪些结论只是推断。
- 对读者下一步有什么建议。

19.1 Research Agent 的典型问题

Research Agent 常见失败：

- 搜索不全。
- 只看摘要不看正文。
- 把营销稿当事实。
- 引用失效链接。
- 没区分一手来源和二手解读。
- 结论比证据更强。
- 忽略反例。
- 把私人讨论直接写进公开报告。

19.2 Research Harness 的基本流程

问题定义

- > 本地/外部资料发现
- > 来源分类
- > 高价值来源展开
- > claim-evidence ledger
- > 观点对比
- > 结论分级
- > 报告审查

19.3 来源分级

等级	类型	用途
Primary	官方文档、论文、源码、原始数据	支撑事实
Strong secondary	深度分析、专家文章	支撑解释
Community	微信文章、论坛、tweet	发现问题和观点
Local discussion	私有聊天、本地记录	发现线索，不直接作为公开事实
Ephemeral	临时文件、私有附件	不公开引用

19.4 Evidence Ledger

研究任务最好维护证据账本：

```
## Evidence Record

### Claim
...

### Evidence
...

### Source
...

### Source Type
primary / secondary / community / local

### Confidence
high / medium / low
```

19.5 Research Harness 的验收

一份研究报告不能只看“写得长不长”。更重要的是：

- 是否覆盖关键来源。
 - 是否有证据链。
 - 是否区分事实和判断。
 - 是否处理反例。
 - 是否说明局限。
 - 是否给出可执行建议。
-

21. Browser / UI Agent Harness

自进化视角：Browser/UI Harness 的进化信号来自截图、DOM、可访问性树、网络请求和用户操作结果。视觉验证不是装饰，而是 feedback signal。

本章学习目标

浏览器 Agent 的难点是现实网页不稳定。本章要让你知道如何给 UI 自动化加可靠性。

UI 任务的四种验证

DOM 验证

检查元素是否存在。

适合：

- 表单字段。
- 按钮。
- 页面标题。

不足：

- 元素存在不代表用户看得见。

Visual 验证

截图检查页面是否符合预期。

适合：

- UI 是否空白。
- 内容是否重叠。
- 移动端布局。

Network 验证

检查请求是否成功。

适合：

- 表单提交。
- API 调用。
- 数据加载。

State 验证

检查业务状态是否改变。

适合：

- 订单是否创建。
- 记录是否保存。
- 设置是否生效。

UI Agent 的稳定策略

- 优先使用可访问名称和 label。

- 避免脆弱 CSS selector。
- 等待明确状态，不盲目 sleep。
- 失败时截图。
- 记录当前 URL。
- 检查错误提示。
- 对登录态做显式判断。

UI Harness 反例

差流程：

打开页面，点击提交，如果没有报错就算成功。

好流程：

打开页面
确认登录态
等待目标表单可见
填写字段
提交
等待成功 toast 或目标 API 返回 200
刷新页面确认数据存在
保存截图和网络结果

20.1 浏览器 Agent 的典型问题

浏览器任务很容易失败：

- 页面动态加载。
- DOM 结构变化。

- 按钮不可见。
- 登录状态不同。
- 网络慢。
- 元素选择器不稳定。
- 页面看起来成功，实际提交失败。

20.2 UI Harness 需要什么

Browser / UI Harness 至少需要：

- 明确目标页面。
- 稳定选择器策略。
- 截图或视觉检查。
- 网络请求观察。
- 表单提交确认。
- 错误状态识别。
- 重试和超时策略。
- 登录状态管理。

20.3 视觉验证

很多 UI 任务不能只看 DOM。需要截图或视觉检查：

- 页面是否空白。
- 元素是否重叠。
- 按钮是否可见。

- 移动端是否溢出。
- 关键图片是否加载。

20.4 UI Harness 的反模式

只写：

打开页面，点按钮，看看是否成功。

这太脆弱。

更好的协议：

1. Navigate to URL.
 2. Wait for network idle or target selector.
 3. Verify page title and main region.
 4. Fill form using stable labels.
 5. Submit.
 6. Wait for success indicator.
 7. Capture screenshot.
 8. Check no visible error message.
 9. Record URL, screenshot path, and result.
-

22. Data Agent Harness

自进化视角：Data Harness 的核心是口径。任何口径纠正、计数不一致、字段缺失，都应该沉淀成 metric definition、reconciliation check 或 schema rule。

本章学习目标

数据任务最怕“口径错但格式对”。本章要让你学会设计 count-first、provenance-first 的数据 Harness。

数据任务的三条铁律

第一，原始数据不可变。

所有转换都应该从原始输入派生，不能覆盖原始文件。

第二，口径先于计算。

先定义业务口径，再写处理逻辑。

第三，每个输出都能追溯。

输出里的每一行，最好能回到输入来源。

Count Reconciliation

数据任务必须做 count reconciliation。

示例：

```
Input rows: 10,000
Rows with valid phone: 9,820
Unique phones: 9,100
Duplicate phones: 720
Output rows: 9,100
Rejected rows: 180
```

如果这些数字对不上，不要急着交付。

数据口径卡片

Metric Definition

Name: unique_customer_count

Definition:

Count unique normalized phone numbers after
removing empty and invalid phones.

Dedupe key:

normalized_phone

Time range:

2026-05-01 00:00:00 to 2026-05-19 23:59:59 local
time

Exclusions:

test records, invalid phone, blank owner

Data Harness 的验证

- schema 检查。
- 空值检查。
- 唯一键检查。
- count 对齐。
- 抽样复核。
- 输出文件打开检查。
- 业务口径复述。

数据任务里，Agent 最后应该报告数字，而不是只说“已生成”。

21.1 数据任务的典型问题

Data Agent 常见失败：

- count 不一致。
- 去重口径错误。
- 字段类型误判。
- 时间窗口错。
- 混用业务口径。
- 空值处理不清。
- 输出表无法复查。

21.2 数据 Harness 的核心

数据任务必须强调可追溯：

- 输入文件。
- 字段含义。
- 清洗规则。
- 过滤条件。
- 去重口径。
- count 校验。
- 输出路径。
- 异常记录。

21.3 数据任务 Checklist

Data Task Checklist

- [] Record input files.
- [] Inspect schema.
- [] Define business counts.
- [] Define dedupe keys.
- [] Preserve raw data.
- [] Write transformation steps.
- [] Validate totals.
- [] Export reproducible artifact.

21.4 数据 Harness 的原则

数据任务里，“看起来差不多”是不可接受的。每个 count 都要有口径，每个输出都要能回到输入。

23. Long-running Agent Harness

自进化视角：长任务需要保存状态、决策和未完成事项。没有 durable state，Agent 每次恢复都是重启，不是进化。

本章学习目标

长任务 Harness 的核心是状态管理。本章要让你学会把一个无法一次完成的大任务拆成可恢复、可交接、可验证的阶段。

长任务失败的本质

长任务失败通常不是某一步做不了，而是状态丢了。

状态包括：

- 原始目标。
- 已完成内容。
- 当前假设。
- 已经排除的方案。
- 中间产物。
- 风险。

- 下一步。

如果这些只存在对话上下文里，一旦上下文变长或 session 中断，就会丢。

Sprint Contract

长任务可以拆成 sprint，每个 sprint 有 contract：

```
## Sprint Contract
```

```
Goal:
```

```
Inputs:
```

```
Allowed scope:
```

```
Expected artifact:
```

```
Verification:
```

```
Handoff required:
```

Sprint Contract 防止 Agent 在长任务中无限扩散。

Planner / Generator / Evaluator

长任务中，角色分离很重要。

Planner 不应该直接沉迷实现。

Generator 不应该独自决定质量达标。

Evaluator 不应该改目标。

Integrator 不应该忽略未解决风险。

角色分离不是形式主义，它降低自评偏差。

Resume 能力

一个长任务 Harness 至少应该支持：

- 从 run log 恢复。
- 从 handoff 恢复。
- 从 artifact 恢复。
- 从 open questions 恢复。

如果不能恢复，就不是可靠长任务 Harness。

22.1 长任务为什么难

长任务难在：

- 上下文窗口不够。
- session 会中断。
- 目标会漂移。
- 中间状态复杂。
- Agent 会忘记之前决定。
- 人也会忘记为什么走到这一步。

22.2 长任务的基本模式

常见模式：

Planner -> Worker -> Evaluator -> Integrator

Planner:

- 拆解目标。
- 定义阶段。
- 写任务边界。

Worker:

- 执行具体任务。
- 产出 artifact。

Evaluator:

- 检查结果。
- 找漏洞。
- 要求返工。

Integrator:

- 合并结果。
- 处理冲突。
- 形成最终交付。

22.3 长任务的状态包

长任务必须有状态包：

Project State

Objective

...

Current Phase

...

Completed Artifacts

...

Decisions

...

Known Risks

...

Blockers

...

Next Step

...

22.4 什么时候拆任务

需要拆任务的信号：

- 任务超过一个上下文窗口。
- 同时涉及多个模块。
- 需要不同专业能力。
- 需要长时间等待外部结果。
- 需要多轮验证。
- 中间 artifact 可以独立验收。

24. Multi-agent Harness

自进化视角：Multi-agent Harness 不是多放几个 Agent，而是让分工、交接、审查和归因可进化。否则多 Agent 只会放大混乱。

本章学习目标

多 Agent Harness 解决并行和专业分工，但也带来协调复杂度。本章要让你知道什么时候应该多 Agent，什么时候不应该。

多 Agent 的价值

多 Agent 有三个真实价值：

1. 并行探索不同信息源。
2. 分离执行和审查。
3. 让不同角色使用不同上下文和工具。

例如研究任务：

- Agent A 查官方来源。
- Agent B 查开源项目。
- Agent C 查中文社区。

- Agent D 做证据审查。

比一个 Agent 顺序查更快，也更容易发现偏差。

多 Agent 的成本

多 Agent 会增加：

- 上下文传递成本。
- 冲突解决成本。
- 重复劳动。
- 结果整合难度。
- token 和工具成本。

所以多 Agent 不是默认选项。

Ownership

多 Agent 一定要有 ownership。

代码任务 ownership：

```
Worker A owns backend files.  
Worker B owns frontend files.  
Reviewer only comments, does not edit.  
Integrator resolves conflicts.
```

研究任务 ownership：

```
Researcher A owns primary sources.  
Researcher B owns community sources.
```

Researcher C owns GitHub projects.
Synthesizer owns final narrative.
Reviewer owns claim-evidence check.

没有 ownership, 多 Agent 会互相踩踏。

合并协议

多 Agent 结果需要合并协议：

- 每个 Agent 输出同样格式。
- 必须标注来源和置信度。
- 冲突观点不能自动抹平。
- Integrator 负责最终取舍。
- Reviewer 检查遗漏和重复。

23.1 多 Agent 不是越多越好

多 Agent 可以并行，但也会增加协调成本。

适合多 Agent 的情况：

- 子任务独立。
- 写入范围不重叠。
- 可以并行查不同来源。
- reviewer 可以独立检查。

不适合多 Agent 的情况：

- 任务边界不清。
- 所有人都改同一文件。

- 主任务依赖某个结果才能继续。
- 协调成本超过执行成本。

23.2 多 Agent 的角色设计

常见角色：

角色	职责
Planner	拆解任务和定义边界
Explorer	查代码、查资料、回答具体问题
Worker	做具体修改或产出
Reviewer	找问题和风险
Integrator	合并结果并做最终判断

23.3 多 Agent 的协议

每个 Agent 都应该有：

- 明确任务。
- 输入范围。
- 输出格式。
- 禁止事项。
- 是否可以改文件。
- 文件 ownership。
- 完成标准。

不要让多个 Agent 用同一个模糊目标各自发挥。

25. 从零设计一个自进化 Harness

本章主线：从零设计 Harness 时，v0 就要预留 evolution log、failure review 和 eval seed。不要等系统失控后再补闭环。

本章学习目标

这一章是实战核心。你要学会从一个真实任务出发，设计最小可用 Harness，而不是先搭大框架。

完整设计流程

可以用十步法：

1. 选择任务
2. 描述目标
3. 列出失败模式
4. 设计上下文
5. 设计工具
6. 设计权限
7. 设计 workflow
8. 设计验证
9. 设计日志
10. 跑一次并复盘

示例：设计 Research Harness

任务：

研究一个新兴技术概念，并输出学习小册子。

Step 1：目标

目标：让读者看一篇文档就能理解并开始实践该概念。

Step 2：失败模式

失败模式	后果
只复述社区文章	结论不可靠
漏掉官方来源	定义不准
没展开 URL	只做二手总结
内容太浅	读者无法学习
没有练习	无法落地
泄露私有讨论	隐私风险

Step 3：上下文

Required:

- topic
- local discussion summary
- URL frequency table
- primary source expansions
- community source summaries

Excluded:

- raw private chat logs

- private attachment URLs
- personal IDs

Step 4: 工具

Tools:

- local search
- URL extractor
- web reader
- GitHub reader
- video metadata reader
- Markdown writer
- privacy scanner

Step 5: workflow

local discovery

- > URL extraction
- > source expansion
- > evidence ledger
- > outline
- > handbook draft
- > review
- > final

Step 6: 验证

Checks:

- top recurring URLs expanded or explicitly classified
- no personal IDs
- every major claim has source class
- includes examples and exercises
- includes templates

示例：设计 Coding Harness

任务：

修复一个 CLI help 行为 bug。

最小 Harness：

Task Spec:

- Bare command should show help.
- Do not query data without explicit filter.

Context:

- CLI command file.
- Existing tests.
- AGENTS rules.

Tools:

- rg
- file read
- apply_patch
- go test

Verification:

- go test ./...
- manual smoke: command with no args
- inspect diff

从 v0 到 v1

v0 Harness 可以很粗，只要有闭环。

v1 再把重复步骤脚本化。

v2 加 eval。

v3 加长期记忆和多 Agent。

不要试图一步到位。

24.1 第一步：选一个具体任务

不要从“我要设计一个万能 Harness”开始。先选一个具体任务。

好任务：

- 高频。
- 有明确输入输出。
- 有失败样本。
- 有验证方式。
- 对你有真实价值。

例如：

- 修复代码 bug。
- 从本地聊天生成研究报告。
- 自动整理一批表格。
- 浏览器填写固定表单。
- 监控某个网页变化。

24.2 第二步：写任务规格

用这个模板：

```
## Task
```

```
### Goal
```

- ### Inputs
- ### Scope
- ### Non-goals
- ### Risks
- ### Acceptance Criteria

24.3 第三步：列失败模式

问自己：

- Agent 最可能在哪一步错？
- 错了以后损失是什么？
- 人怎么发现？
- 能不能自动发现？
- 能不能提前避免？

示例：

失败模式	预防	检查
漏掉重要 URL	高频 URL 排序	URL coverage eval
输出私人 ID	脱敏规则	privacy scanner
结论无证据	evidence ledger	claim grounding check
跑错测试	固定命令	test log check

24.4 第四步：设计最小闭环

最小闭环要包含：

输入 -> 执行 -> 验证 -> 记录 -> 改进

不要先做复杂 UI、复杂 Agent 编排。先让闭环跑起来。

24.5 第五步：把经验固化

每次失败后，决定改哪里：

- 改 task spec。
 - 改 context pack。
 - 改 tool wrapper。
 - 改 permission policy。
 - 加 eval。
 - 加 skill。
 - 加 memory。
-

26. 如何诊断 Harness：从失败定位到进化候选

本章主线：诊断不是为了写复盘，而是为了产生改进候选。每次失败都要定位到 Harness 层级，并决定是改规格、上下文、工具、权限、记忆、技能、流程、eval 还是日志。

本章学习目标

本章要让你获得一个重要能力：Agent 失败后，不急着换模型，而是系统性定位 Harness 哪一层失败。

失败诊断的基本原则

不要问：

为什么 AI 又不行？

要问：

它在哪个环节获得了错误信息、采取了错误动作、缺少了验证，或者没有被及时纠正？

这句话会把情绪转化为工程问题。

八层定位法

1. Spec Failure

症状：

- Agent 做了不该做的事。
- 结果和你想的不一样。
- Agent 反复问澄清问题。

修复：

- 写清 Goal。
- 写 Scope 和 Non-goals。
- 增加 Acceptance Criteria。

2. Context Failure

症状：

- Agent 引用旧信息。
- 漏掉关键文件。
- 误解业务口径。

修复：

- 明确 Source of Truth。
- 建 Context Pack。
- 删除过期上下文。

3. Tool Failure

症状：

- Agent 用错工具。
- 工具输出太长。
- 错误信息不可读。

修复：

- 加 wrapper。
- 增加 JSON 输出。
- 写工具选择规则。

4. Permission Failure

症状：

- Agent 做了危险动作。
- Agent 因权限不足无法继续。
- 审批频率过高。

修复：

- 分级权限。
- 明确 approval gate。
- 对常用低风险动作放权。

5. Workflow Failure

症状：

- Agent 跳步。
- 顺序错。
- 没有中间产物。

修复：

- 写 workflow。
- 加阶段检查点。
- 要求 handoff。

6. Verification Failure

症状：

- Agent 假完成。
- 结果错但没被发现。
- 测试没有覆盖关键路径。

修复：

- 增加确定性检查。
- 增加 eval。
- 固定验证命令。

7. Observability Failure

症状：

- 不知道 Agent 做了什么。
- 无法复现失败。

- 不知道该改哪里。

修复：

- 加 run log。
- 记录工具参数。
- 保存 artifact。

8. Model Capability Failure

症状：

- 前面七层都合理，但任务仍超出模型能力。

修复：

- 换模型。
- 拆任务。
- 增加工具。
- 增加人工决策点。

诊断案例

案例：Agent 写了一份研究报告，但内容很浅。

不要只说“写得不够深入”。逐层分析：

层	可能问题
Spec	没定义目标读者和深度
Context	只给了少量来源
Tool	没有展开 URL 的工具
Workflow	跳过 evidence ledger
Verification	没有检查是否包含例子和练习
Observability	不知道它实际读了哪些来源

修复后：

- Task Spec 加“读者看完能独立设计 Harness”。
- Workflow 加 source expansion。
- Eval 加“每章必须有例子、反例、练习或模板”。

这就是 Harness 诊断。

25.1 诊断顺序

Agent 失败时，不要直接说“模型不行”。按顺序查：

1. 任务是否模糊。
2. 上下文是否错误或缺失。
3. 工具是否不可用或输出难懂。
4. 权限是否过大或过小。

- 5. 工作流是否跳步。
- 6. 验证是否缺失。
- 7. 日志是否足以复盘。
- 8. 模型能力是否确实不足。

很多失败是 Harness 失败，不是模型失败。

25.2 常见症状和原因

症状	可能原因
Agent 做了范围外改动	task scope 不清或权限过大
Agent 反复试错	缺少项目地图或工具反馈差
Agent 自信但错误	缺少验证
Agent 每次犯同样错误	缺少 memory/skill
Agent 越做越乱	任务过大，缺少 handoff
Agent 成本过高	上下文太大，缓存差，工具输出过长
Agent 不敢行动	权限太窄或规格太模糊

25.3 复盘模板

```
## Harness Failure Review

### What happened

### Expected behavior

### Actual behavior
```

Where did it fail

- Spec
- Context
- Tool
- Permission
- Workflow
- Verification
- Observability
- Model capability

Evidence

Fix

New rule / eval / skill

27. Harness 成熟度模型：从静态可用到自进化

本章主线：成熟度模型从“是否能完成任务”升级为“是否能从任务中变好”。一个静态可用的 Harness，等级仍然有限。

本章学习目标

成熟度模型不是用来炫耀等级，而是帮助你判断下一步该补什么。很多团队卡住，是因为在 Level 1 的基础上幻想 Level 5 的效果。

Harness 成熟度模型

成熟度不是自动化越多越好，而是风险、验证和进化能力是否匹配。



各级的核心差异

等级	核心能力	最大短板
Level 0	模型能回答	不可控
Level 1	Agent 能行动	不稳定
Level 2	有规则和验证	不会持续学习
Level 3	有记忆和技能	度量不足
Level 4	有 eval 和闭环	组织推广成本
Level 5	组织级系统	治理复杂

如何判断自己的等级

问十个问题：

- 1. 有没有固定任务规格？
- 2. 有没有上下文入口？
- 3. 有没有工具权限规则？
- 4. 有没有自动验证？
- 5. 有没有 run log？
- 6. 有没有失败复盘？
- 7. 有没有可复用 skill？
- 8. 有没有长期 memory？
- 9. 有没有 golden tasks？
- 0. 有没有指标证明变好？

大致判断：

- 0 到 2 个是 Level 0-1。
- 3 到 5 个是 Level 2。
- 6 到 8 个是 Level 3。
- 9 到 10 个才接近 Level 4。

升级顺序

不要从 Level 1 直接跳 Level 5。推荐：

```
先补 task spec
再补 verification
再补 run log
再补 skill
再补 memory
再补 eval
再补 multi-agent
```

因为没有验证和日志，后面所有优化都很难证明有效。

组织级 Harness 的标志

组织级 Harness 不只是个人工作流，而是：

- 团队共享规则。
- 任务类型标准化。
- 权限和审计统一。
- eval 持续运行。
- 失败进入知识库。
- Agent 产出可以进入正式流程。

这需要工程、产品、安全、运营共同设计。

Level 0：裸模型

特征：

- 只靠聊天。
- 没有工具。
- 没有验证。
- 结果靠人判断。

适合：

- 头脑风暴。
- 简单解释。
- 一次性草稿。

Level 1：有工具的 Agent

特征：

- 能读文件、跑命令、查网页。
- 有基本任务执行能力。
- 但验证和规则不稳定。

适合：

- 小范围自动化。
- 简单代码修改。
- 资料收集。

Level 2: 有规则和验证

特征：

- 有项目规则。
- 有任务规格。
- 有测试或检查。
- 有权限边界。

适合：

- 日常工程协作。
- 可控的数据处理。
- 较稳定研究任务。

Level 3: 有记忆和技能

特征：

- 能复用经验。
- 有固定 skill。
- 有失败复盘。
- 有部分自动 eval。

适合：

- 重复任务。
- 团队级 Agent 工作流。
- 复杂研究和代码任务。

Level 4: 有长期闭环

特征：

- 有 run log。
- 有 golden tasks。
- 有持续 eval。
- 有版本化 Harness。
- 能度量改进。

适合：

- 生产级 Agent 系统。
- 多人协作。
- 长期运营。

Level 5: 组织级 Harness

特征：

- Harness 成为组织流程。
- 权限、审计、合规、成本、质量全部纳入系统。
- 多 Agent、多团队、多任务共享基础设施。

适合：

- 大规模 Agent-driven development。
- 企业内部自动化。

- 专业 Agent 产品。
-

28. 常见反模式：伪自进化、规则堆积和漂移

本章主线：反模式有一个关键判断标准：凡是不能被验证、不能被回滚、不能减少重复失败的“进化”，都只是伪进化。

本章学习目标

反模式比最佳实践更有教育意义。因为很多 Harness 不是没做，而是做偏了。

反模式 1：把 Harness 当成文档仓库

症状：

- 写了很多规则。
- Agent 仍然经常错。
- 没有自动检查。
- 没有反馈更新。

问题：

文档不会自动执行。规则必须进入流程、工具或验证。

修复：

- 把高频规则变成 checklist。
- 把可检查规则变成脚本。
- 把流程规则变成 workflow。

反模式 2：把所有问题都推给模型

症状：

- 失败就换模型。
- 换模型后仍然同类失败。

问题：

很多问题是上下文、工具、验证缺失。

修复：

- 先跑八层诊断。
- 确认不是 Harness 问题后再换模型。

反模式 3：只追求自动化率

症状：

- 尽量不让人介入。
- Agent 能自动做很多事。
- 但错误影响变大。

问题：

自动化率不是唯一指标。高风险任务需要人类控制点。

修复：

- 按风险分级。
- 保留 approval gate。
- 自动化低风险重复动作。

反模式 4：Eval 表演

症状：

- 有很多评分。
- 分数看起来不错。
- 实际任务仍然失败。

问题：

Eval 没贴近真实失败模式。

修复：

- 从真实失败样本构建 golden tasks。
- Eval 指标绑定业务风险。
- 定期检查 eval 是否过拟合。

反模式 5：Context 大杂烩

症状：

- 什么都塞给 Agent。
- Agent 输出混乱。
- 成本很高。

问题：

上下文没有优先级和生命周期。

修复：

- 建 Source of Truth。
- 上下文分 required / optional / forbidden。
- 做摘要和引用。

反模式 6：多 Agent 混战

症状：

- 开了很多 Agent。
- 输出互相重复或冲突。
- 集成更累。

问题：

没有 ownership 和合并协议。

修复：

- 只有独立任务才并行。
- 每个 Agent 有明确输出格式。
- 设置 Integrator。

27.1 Prompt 崇拜

认为只要 prompt 写得足够好，Agent 就能稳定完成任务。

修正：

Prompt 只是入口。真实可靠性来自工具、验证、权限、日志和反馈。

27.2 规则堆积

不断往系统里加规则，但不清楚每条规则解决什么问题。

修正：

每条规则都要绑定失败模式。不能解释价值的规则应该删除。

27.3 无验证交付

Agent 说完成就算完成。

修正：

完成必须绑定验证。不能验证的地方要明确风险。

27.4 把所有判断交给 LLM judge

本来可以程序检查的东西，却交给另一个模型判断。

修正：

确定性检查优先。LLM judge 只处理语义和主观质量。

27.5 上来就多 Agent

任务还没拆清楚，就派多个 Agent 并行。

修正：

先定义 ownership、输入输出和集成点。否则并行会制造更多混乱。

27.6 忽略人类体验

Harness 设计得很完整，但人类使用成本太高。

修正：

Harness 应该减少人的认知负担，不是制造更多表格和流程。

27.7 不更新 Harness

模型变了、项目变了、任务变了，但 Harness 仍然不变。

修正：

Harness 需要版本化和定期清理。过期规则会拖累系统。

29. 七天学习和实践路线：每天建立一个进化闭环

本章主线：七天路线不再只是搭建一个 Harness，而是每天补一段进化闭环，最后得到能记录失败、生成候选、验证升级的最小系统。

本章学习目标

这不是阅读计划，而是实践计划。七天后你应该拥有一个最小 Harness，而不只是理解概念。

选择你的练习任务

建议选择一个真实但风险可控的任务。

好选择：

- 写一份研究报告。
- 修一个小 bug。
- 整理一批 CSV。
- 自动检查网页。
- 生成周报。

不建议第一周选择：

- 生产数据库操作。
- 自动发客户消息。
- 大规模重构。
- 涉及付款或合规的流程。

每天的完成标准

Day 1 完成标准：

- 能用自己的话解释 Harness。
- 能指出一个失败案例属于哪一层。

Day 2 完成标准：

- 有一份 Task Spec。
- 有一个 Context Pack。

Day 3 完成标准：

- 有工具清单。
- 有权限分级。

Day 4 完成标准：

- 有验证清单。
- 有 run log 模板。

Day 5 完成标准：

- 有一个 Skill 草稿。
- 有一条可复用 memory。

Day 6 完成标准：

- 用 Harness 跑一次真实任务。
- 保存 artifact。

Day 7 完成标准：

- 做失败复盘。
- 写下一版 Harness 改进项。

七天后你应该得到什么

最终产物应该是：

```
HARNESS.md
Task Spec
Context Pack
Tool Policy
Verification Checklist
Run Log
One Skill
One Failure Review
Next Improvement
```

这就是你的第一个 Harness。

Day 1: 建立概念

阅读：

- 第 1 至 6 章。

任务：

- 用一句话定义 Harness。
- 写出 Prompt、Context、Tool、Eval、Harness 的区别。
- 找一个你最近使用 Agent 的失败案例，判断失败发生在哪一层。

输出：

我对 Harness 的定义：

最近一次 Agent 失败：

失败层级：

如果有 Harness，应该补什么：

Day 2：任务规格和上下文

阅读：

- 第 7 至 8 章。

任务：

- 选一个真实任务，写 Task Spec。
- 写 Context Pack。

输出：

Task Spec:

Context Pack:

Non-goals:

Acceptance Criteria:

Day 3: 工具和权限

阅读:

- 第 9 至 10 章。

任务:

- 列出这个任务需要的工具。
- 给每个工具标注权限级别。
- 写出哪些动作需要人工批准。

输出:

Tools:

Permission:

Approval Required:

Forbidden:

Day 4: 验证和日志

阅读:

- 第 14 至 15 章。

任务：

- 为任务设计验证步骤。
- 写 run log 模板。
- 区分哪些检查是确定性的，哪些需要人工或 LLM。

输出：

Verification:

Deterministic Checks:

Semantic Checks:

Run Log:

Day 5: 技能和记忆

阅读：

- 第 11 至 12 章。

任务：

- 把任务里可复用的一步写成 Skill。
- 写一条未来有价值的 memory。

输出：

Skill:

When to use:

Steps:

Output:

Memory:

Day 6: 构建最小 Harness

阅读:

- 第 24 至 25 章。

任务:

- 把前几天的规格、上下文、工具、权限、验证、日志拼成一个最小 Harness。
- 用它跑一次真实任务。

输出:

Harness v0:

Run Result:

Failures:

Fixes:

Day 7: 复盘和升级

阅读:

- 第 26 至 27 章。

任务：

- 判断你的 Harness 处于成熟度哪一级。
- 写出下一版要加的一项 eval 或 skill。

输出：

Current Maturity Level:

Biggest Weakness:

Next Improvement:

How to Measure:

30. 模板库：把经验变成可进化资产

本章主线：模板库现在承担“可进化资产库”的角色。模板不只是输出格式，而是让经验被审计、验证、提升和清理的结构。

本章学习目标

模板不是形式，而是把思考流程固化下来。本章的模板可以直接复制到自己的项目里，但不要机械填写。每个模板都应该和真实任务绑定。

模板使用原则

1. 先填最小字段，不要为了完整而完整。
2. 每个字段都应该能影响 Agent 行为。
3. 如果某个字段连续三次没用，就删掉。
4. 如果某类错误重复出现，就加字段或加检查。
5. 模板要版本化。

一个模板如何演进

第一版 Task Spec 可能只有：

Goal
Scope
Output

当 Agent 经常做多余事情，加：

Non-goals

当 Agent 经常假完成，加：

Acceptance Criteria
Verification

当 Agent 经常触碰风险动作，加：

Approval Required
Forbidden

模板应该从失败中长出来。

29.1 HARNESS.md 模板

```
# Harness: <Name>
```

```
## Purpose
```

```
This harness helps agents complete <task type>  
reliably.
```

Scope

Included:

- ...

Excluded:

- ...

Inputs

- ...

Context Pack

Required:

- ...

Optional:

- ...

Do not include:

- ...

Tools

Tool	Purpose	Permission	Notes
---	---	---	---
...	...	...	...

Workflow

1. ...
2. ...
3. ...

Verification

- ...

Human Approval

Requires approval:

— ...

Forbidden:

— ...

Output

— ...

Run Log

Each run must record:

— ...

Known Failure Modes

Failure	Prevention	Detection	Fix	
---	---	---	---	
...	...	...	...	

29.2 Task Spec 模板

Task Spec

Goal

Background

Scope

Non-goals

Constraints

Inputs

Acceptance Criteria

Required Output

```
## Risks
```

29.3 Context Pack 模板

```
# Context Pack
```

```
## Task
```

```
## Required Context
```

```
## Optional Context
```

```
## Source of Truth
```

```
## Outdated or Risky Context
```

```
## Do Not Include
```

29.4 Skill 模板

```
# Skill: <Name>
```

```
## When to use
```

```
## Inputs
```

```
## Steps
```

```
## Tools
```

```
## Output
```

```
## Verification
```

```
## Safety
```


Common Failures

29.5 Eval Card 模板

Eval: <Name>

Purpose

Task Type

Input

Expected Behavior

Failure Conditions

Scoring

Score	Meaning
---	---
0	Failed
1	Partial
2	Good

Deterministic Checks

Semantic Checks

Golden Examples

29.6 Run Log 模板

Run Log

Run ID

Date

Task

Inputs

Tools Used

Steps

Observations

Failures

Verification

Output

Learnings

Harness Changes Needed

29.7 Handoff 模板

Handoff

Objective

Current State

Completed Work

Decisions Made

Artifacts

Open Questions

Risks

Next Step

29.8 Evidence Ledger 模板

Evidence Ledger

Claim	Evidence	Source	Type	Confidence
	Caveat			
---	---	---	---	---
...	...	...	primary / secondary / local	
		high / medium / low	...	

29.9 Privacy Checklist

Privacy Checklist

- [] No secrets.
- [] No full tokens or keys.
- [] No private personal IDs.
- [] No private attachment URLs.
- [] No unnecessary raw chat logs.
- [] Private sources are summarized, not exposed.
- [] Public claims use public sources.

30.10 Evolution Record 模板

Evolution Record

ID

EV-YYYY-MM-DD-001

Trigger

这次进化由什么触发：失败、用户纠正、eval 失败、工具错误、成本异常、成功模式？

Evidence

列出证据，不写空泛判断。

Failure Layer

Task Spec / Context / Tool / Permission / Memory
/ Skill / Workflow / Eval /
Observability

Proposed Change

具体要改什么。

Patch Scope

影响哪些文件、规则、skill、工具或流程。

Verification

如何证明这个改动让系统变好。

Promotion Criteria

什么条件下进入正式 Harness。

Rollback Criteria

什么条件下撤回。

Status

Candidate / Promoted / Rolled back / Archived

30.11 Memory Item 模板

Memory Item

Statement

需要保存的稳定经验。

Scope

适用项目、任务类型或用户偏好。

Evidence

来自哪次任务、哪次纠正或哪份报告。

Confidence

Low / Medium / High

Expiry / Review

什么时候复查，什么时候失效。

Status

Candidate / Active / Scoped / Archived

30.12 Source Note 模板

Source Note

Title

资料标题。

Source URL

来源链接。

Problem

这篇资料解决什么失败。

Feedback Signal

它依赖什么反馈：测试、reward、工具、人类、模型自评？

Evolvable Object

它进化什么：prompt、memory、tool、workflow、agent
architecture、code？

Evolution Loop

它的闭环是什么。

Evaluator

它如何判断变好了。

Safety Boundary

它如何防止坏改动。

Harness Translation

我能把它翻译成什么工程动作。

30.13 Rule Prune Review 模板

Rule Prune Review

Rule

要清理的规则。

Why It Was Added

当初为什么加入。

Last Triggered

最后一次有效触发是什么时候。

Still Relevant

yes / no / uncertain

Risk If Removed

删除后可能有什么风险。

Action

keep / revise / archive / delete

31. 术语表

本章学习目标

术语不是为了显得专业，而是为了避免团队沟通混乱。很多争论来自同一个词被不同人用来指不同东西。

学习术语的方法

不要死记定义。每个术语都问三个问题：

- 它解决什么问题？
- 它不解决什么问题？
- 它在 Harness 里属于哪一层？

例如 Eval：

- 解决：判断输出质量。
- 不解决：自动提供上下文。
- 所属层：验证和评测。

容易混淆的词

Harness vs Framework

Framework 是工具或代码框架。Harness 是运行环境和工程结构。Framework 可以承载 Harness，但 Harness 不等于 Framework。

Skill vs Tool

Tool 是可调能力。Skill 是使用工具完成任务的过程。

例如：

- Tool：浏览器。
- Skill：展开网页、提取正文、标记失败、写摘要。

Memory vs Context

Memory 是长期保存的信息。Context 是当前任务里实际给模型看的信息。Memory 需要被检索、筛选后才进入 Context。

Eval vs Review

Eval 更标准化、可重复。Review 更灵活，常由人或 LLM 做主观判断。

Workflow vs Protocol

Workflow 是正常路径。Protocol 包括异常处理和交互规则。

Agent

能围绕目标进行多步推理、调用工具、观察结果并继续行动的系统。

Agent Loop

Agent 的基本循环：思考、行动、观察、继续或结束。

Harness

围绕 Agent 的运行环境和工程约束，包括上下文、工具、权限、验证、日志、记忆和反馈。

Prompt Engineering

设计模型输入指令的实践。

Context Engineering

选择、组织和更新模型可见信息的实践。

Tool Use

让模型调用外部工具，如 shell、API、browser、database。

Skill

可复用的任务过程，通常包含触发条件、步骤、工具、输出和验证。

Memory

外部化保存的长期信息，用来跨 session 保持项目规则、用户偏好、历史决策和失败经验。

Eval

用于评估 Agent 输出或行为质量的任务、规则、测试或评分机制。

Golden Task

固定的代表性任务，用来比较 Harness 改动前后的效果。

Handoff

长任务中跨 Agent 或跨 session 传递状态的结构化 artifact。

Human-in-the-loop

人在 Agent 流程中承担目标设定、风险审批、审查和接管的机制。

Observability

记录和理解 Agent 行为的能力，包括日志、trace、工具输出、错误和指标。

项目插章：这些项目补足了小册子的工程盲区

项目解剖方法

读 Harness 项目时，不要只看它用了什么模型，也不要只看 README 里的宣传语。要把它放到九个可进化面里拆：任务规格、上下文、工具、权限、记忆、技能、 workflows、验证、可观测性，再看它如何从失败中改进。

Project Harness Review

Project

Primary Task

这个项目把哪类 Agent 任务 harness 起来？

Agent Boundary

Agent 能做什么，不能做什么？

Tool Surface

它暴露了哪些工具？这些工具是否有 wrapper、schema、preflight、redaction？

Permission Model

哪些动作自动执行，哪些动作需要人工批准？

Workflow / Gates

任务如何分阶段？是否有 gate、checkpoint、handoff、rollback？

Verification

它如何知道任务完成？测试、benchmark、截图、用户确认、LLM judge、规则检查？

Observability

它是否保存 trace、trajectory、log、artifact?

Evolution Mechanism

它如何从失败中生成规则、skill、eval 或工具改进?

Missing Pieces

它缺什么?

Borrowable Ideas

你自己的 Harness 能借走什么?

案例一：wow-harness

wow-harness 最适合作为“工程治理型 Harness”的案例。它把 Harness 从文档层推进到机制层：hook、gate、validator、只读审查、状态机、fail-closed。

它最值得学的不是“用了多少个 Agent”，而是这几个工程判断：

1. 规则要机械化。只写“不要乱改”是不够的，要用 hook 和 validator 让错误行为被拦住。
2. 审查要隔离。负责审查的 Agent 最好没有写权限，否则审查本身会变成同谋。
3. 状态要推进。复杂任务不能靠聊天上下文自然流动，要有显式 gate。
4. 失败要停住。一个 Harness 宁可 fail closed，也不要再在证据不足时继续推进。

对应小册子的章节：

项目机制	对应 Harness 层
Hook	工具 / 权限 / 工作流
Validator	验证 / 权限
State machine	工作流
Read-only review	Human-in-loop / 多 Agent
Failure-derived invariant	自进化 / 记忆 / 技能

如果你做 Coding Agent Harness, **wow-harness** 的关键启发是：不要把“请小心”写进 prompt，而要把小心变成机制。

案例二：browser-harness

browser-harness 适合学习 Browser Agent 的现实复杂性。浏览器不是一个稳定 API，而是一个持续变化的环境。页面会改版，选择器会失效，加载速度会变化，登录态会过期，同一个按钮在不同用户状态下可能表现不同。

这类项目的核心不是“能点网页”，而是如何把一次次网页操作变成 domain skill。

可借鉴的设计：

1. 每个网站都可能需要自己的 skill。
2. 成功轨迹要保存，因为它可以变成下次的捷径。

3. 失败轨迹也要保存，因为它能生成 recovery pattern。
4. 视觉、DOM、网络请求都可以成为验证信号。
5. Browser Harness 必须考虑 self-healing，而不是只写固定 selector。

对应小册子的章节：工具、技能、工作流、验证、可观测性。

案例三：LangChain Deep Agents

Deep Agents 一类项目展示的是 runtime 层。小册子里说 Harness 是系统，不是文档；runtime 就是这个判断的落地。

它提醒我们：真正跑长任务时，你需要的不只是 prompt，还需要：

- 文件系统。
- 子 Agent。
- 持久状态。
- checkpoint。
- human approval。
- trace。
- deployment。
- 可恢复执行。

如果一个 Harness 没有运行时支撑，它可能只适合短任务。对于长任务、多 Agent 和生产场景，runtime 本身就是 Harness 的一部分。

案例四：SWE-agent / mini-SWE-agent

SWE-agent 系列适合学习 coding benchmark harness。它把 GitHub issue、代码仓库、工具接口、修复轨迹、测试和 SWE-bench 连接起来。

它的价值在于告诉我们：Coding Agent 的验证信号很强，不应该浪费。

可借鉴的设计：

1. 任务来自真实 issue。
2. Agent 的操作可以保存成 trajectory。
3. patch 可以被测试验证。
4. benchmark 可以做横向比较。
5. 失败样本可以回放，形成新的 eval。

对个人 Coding Harness 来说，这意味着最终回答里说“已完成”不重要；重要的是 diff、测试、CI、review 和回放证据。

案例五：Open Deep Research

Research Agent 的难点不是写作，而是证据链。Open Deep Research 一类项目提示我们，研究型 Harness 要关注：搜索、来源筛选、URL 展开、引用、claim map、报告结构和 benchmark。

可借鉴的设计：

1. 搜索不是结束，而是 evidence pipeline 的开始。
2. 每条关键结论应该能追溯到来源。
3. 来源要分级：论文、官方文档、项目 README、博客、社交媒体、二手摘要。
4. Research Harness 应该有 source coverage eval。
5. 报告生成后要检查是否回答了真实问题，而不是只覆盖关键词。

案例六：self-improving-agent / Evolver

这类项目适合学习最小自进化内核。它们的价值不是“神奇地自动变强”，而是把错误、学习、请求、候选改进变成结构化记录。

可借鉴的设计：

1. 先记录错误，不急自动改系统。

- 2. 把错误归因到具体层级。
- 3. 把高价值错误转成候选。
- 4. 候选要经过验证再推广。
- 5. 进化事件要可审计。

这正是个人 Harness 最容易落地的一条路。

项目补足小册子的盲区

盲区	哪个项目补足	小册子应吸收的内容
规则如何机械化	wow-harness	hook / validator / fail-closed
网页经验如何沉淀	browser-harness	domain skill / self-healing
长任务怎么跑	Deep Agents	runtime / checkpoint / state
coding 如何评测	SWE-agent	benchmark / trajectory / replay
研究如何验真	Open Deep Research	source coverage / claim map
自进化如何起步	self-improving-agent / Evolver	learning log / EvolutionEvent

32. 自进化资料地图：论文、系统文章和实践项目

本章学习目标

这一章不是简单的链接列表，而是告诉你：学习 Harness 自进化时，每类资料应该解决什么问题。

读完本章，你要知道：

1. 哪几篇综述适合作为总框架。
2. 哪些论文解释“反思、修正、工具反馈、上下文进化、架构进化”。
3. 哪些项目可以给个人或团队 Harness 提供实践参考。
4. 这些资料如何映射到第 6 章的工程组件。

先给结论：三篇必读

如果只读三篇，建议按这个顺序：

1. A Survey of Self-Evolving Agents

链接：[A Survey of Self-Evolving Agents: What, When, How, and Where to Evolve on the Path to Artificial Super Intelligence](#)

它适合做“总地图”。这篇综述把 self-evolving agents 按三个问题组织：

What to evolve: 模型、记忆、工具、架构等

When to evolve: 测试时、任务间、长期运行中

How to evolve: reward、文本反馈、单 Agent、多 Agent 等

放进 Harness 语境里，它回答的是：一个 **Agent** 系统到底有哪些可进化面。

2. Beyond Individual Intelligence

链接：[Beyond Individual Intelligence: Surveying Collaboration, Failure Attribution, and Self-Evolution in LLM-based Multi-Agent Systems](#)

它适合做“多 Agent / 组织级 Harness”的总框架。它提出的 LIFE progression 很关键：

Lay the foundation

Integrate agents

Find faults

Evolve

这对 Harness 很重要，因为真正的自进化不是“失败后总结一句”，而是要先能协作、能观测、能归因，然后才能改进。

放进 Harness 语境里，它回答的是：**多 Agent** 系统如何从协作失败中演化。

3. Agentic Context Engineering

链接：[Agentic Context Engineering: Evolving Contexts for Self-Improving Language Models](#)

它最贴近日常工程。普通团队很少直接训练模型，但经常会改 prompt、memory、project rules、playbook、skill。ACE 的价值是把这些上下文当成会进化的 playbook，而不是一次性提示词。

它特别提醒两个问题：

brevity bias：总结越来越短，细节丢失

context collapse：多轮改写后，上下文质量坍塌

放进 Harness 语境里，它回答的是：**如何让上下文越用越强，而不是越总结越空。**

资料地图一：反思和自修正

这一组资料解决的是最基础的问题：Agent 失败后，如何利用反馈改进下一次行为。

资料	重点	Harness 启发
Reflexion	用语言反馈和 episodic memory 强化 Agent	failure review 要进入可检索记忆
Self-Refine	生成、反馈、迭代 refine	输出质量可以通过 test-time loop 改善
CRITIC	用工具做 critique 和修正	外部工具反馈比自我感觉可靠
Teaching LLMs to Self-Debug	代码生成后的自调试	Coding Harness 应该有解释、执行、修复循环
Training Language Models to Self-Correct via Reinforcement Learning	用自生成数据训练 self-correction	单纯提示“自我修正”通常不够，需要训练或外部信号
Recursive Introspection	训练模型多轮发现并修正错误	多轮改进能力本身可以成为训练目标

学习时要带着一个问题：

这篇论文里的 feedback signal 是什么？
它来自模型自己、外部工具、环境，还是人类？

Harness 里最可靠的反馈通常来自外部世界：
测试、工具、用户纠正、真实任务结果。

资料地图二： Agent 轨迹和任务学习

这一组资料解决的是： Agent 如何从自己的任务轨迹中学会更好的行动策略。

资料	重点	Harness 启发
AutoAct	自动合成 planning trajectories，形成 sub-agent 分工	可以从有限数据中生成 workflow 样本
LLMs Can Self-Improve At Web Agent Tasks	WebArena 上用自生成数据提升 Web Agent	浏览器 Agent 的轨迹质量需要专门指标
AgentEvolver	self-questioning、self-navigating、self-attributing	自进化要包含任务生成、经验复用、信用分配
ReVeal	code generation 与 self-verification 共同演化	代码 Agent 需要同时提升生成和验证能力

这类资料提醒我们： 运行轨迹不是日志垃圾，而是训练和改进材料。

对 Harness 来说， 要保存的不只是结果， 还包括：

任务目标

中间观察

工具调用

失败动作

修正动作

最终结果

验证结果

用户反馈

没有轨迹，就没有可复盘的自进化。

资料地图三：上下文、记忆和playbook 进化

这一组资料最贴近个人和团队落地。

资料	重点	Harness 启发
Agentic Context Engineering	上下文作为 evolving playbook	memory 和 system prompt 要结构化增量更新
Externalization in LLM Agents	把内部过程外化到工具、记忆、环境	Harness 本质上是把隐性能力外化成系统
self-improving-agent	错误、学习、功能请求进入 .learnings	个人 Agent 可以先从学习日志开始
Evolver	从日志信号生成 GEP 约束提示词和事件	自进化要可审计，不能直接乱改

这类资料可以总结成一句话：

自进化的第一战场不是模型权重，而是上下文资产。

上下文资产包括：

HARNESS.md
AGENTS.md
CLAUDE.md
system prompt

- project rules
- skills
- memory
- eval cases
- workflow templates
- tool wrappers

如果这些资产没有版本、没有来源、没有失效机制，它们就会从“经验”变成“噪声”。

资料地图四：Prompt 和架构搜索

这一组资料更偏研究和高级系统。

资料	重点	Harness 启发
Promptbreeder	进化 task prompt 和 mutation prompt	不只优化答案，也优化“如何优化”
STOP	脚手架程序递归改进自己	自改代码必须有 sandbox 和安全审查
Automated Design of Agentic Systems	meta-agent 自动设计 agent system	Harness 设计空间也可以被搜索
Gödel Agent	自引用 Agent 动态修改自身逻辑	深层自进化是架构级问题，不只是 prompt
Darwin Gödel Machine	archive + 自改代码 + benchmark 验证	进化需要保留多条历史路径，而非单线迭代
Hyperagents	task agent 与 meta agent 合成可编辑代码体	改进机制本身也可能成为进化对象
AlphaEvolve	LLM 生成候选代码，自动 evaluator 筛选	evaluator 是进化系统的核心基础设施

学习这组资料时，要特别警惕一个问题：

它的 evaluator 是什么？

如果某个系统能自我改进，是因为它有明确可测的目标。代码可以跑测试，算法可以打分，Web 任务可以评估完成率，数学问题可以验算。没有 evaluator，进化就没有方向。

资料地图五：Harness 和实践项目

这一组资料不是论文，但对落地很重要。

项目 / 文章	重点	可以借鉴什么
wow-harness	hook、gate、validator、skill、fail-closed governance	用机械约束替代纯文本提醒
self-improving-agent	.learnings 、错误记录、skill 提升	个人级自进化最小实现
Evolver	GEP、Gene、Capsule、EvolutionEvent	可审计的进化资产
Reflexion repo	Reflexion 实验实现	反思记忆如何进入 agent loop
OpenAI: Harness engineering	Harness 概念入口	把 Agent 放进可验证系统
Anthropic: Harness design for long-running agents	长任务 Harness	checkpoint、state、恢复
LangChain: Anatomy of an Agent Harness	Agent Harness 结构	分解 agent 外部工程组件

实践项目要带着怀疑读：

它有哪些机械约束？
它有哪些 eval？
它如何防止误进化？

它是否能回滚？

它是否记录 `evolution event`？

只说“自动成长”的项目，不一定可信。能展示日志、测试、回滚、权限边界和失败样本的项目，才值得认真研究。

从资料到 **Harness** 的映射

下面这张表可以帮助你论文概念翻译成工程动作。

论文/资料概念	Harness 工程动作
verbal reflection	写 failure review
episodic memory	保存 scoped memory
tool-interactive critique	用工具结果做验证反馈
self-debugging	运行、解释、修复代码循环
context playbook	维护结构化 HARNESS.md / skill / rules
mutation prompt	让 Agent 生成改进候选
archive of variants	保留 promoted / archived harness versions
benchmark evaluator	golden task / regression eval
meta-agent	专门负责改进 Harness 的 Agent
self-attribution	失败归因到任务、上下文、工具、权限、流程、eval
open-ended exploration	多候选路线并行，而不是单一路线微调
sandbox	限制自改代码和工具执行范围
EvolutionEvent	每次进化留下结构化审计记录

推荐阅读顺序

不要一上来读最激进的自改代码论文。建议按五轮读。

第一轮：建立工程直觉。

1. OpenAI Harness Engineering。

2. Anthropic long-running agents。
3. LangChain Anatomy of an Agent Harness。
4. wow-harness README。

第二轮：理解反思和反馈。

1. Reflexion。
2. Self-Refine。
3. CRITIC。
4. Self-Debugging。

第三轮：理解上下文进化。

1. Agentic Context Engineering。
2. Externalization in LLM Agents。
3. self-improving-agent。
4. Evolver。

第四轮：理解系统性自进化。

1. A Survey of Self-Evolving Agents。
2. Beyond Individual Intelligence。
3. AgentEvolver。
4. ReVeal。

第五轮：理解架构和程序进化。

1. Promptbreeder。

2. ADAS。
3. STOP。
4. Gödel Agent。
5. Darwin Gödel Machine。
6. Hyperagents。
7. AlphaEvolve。

阅读笔记模板

读每篇论文或项目时，用同一个模板记录，方便转成 Harness 改进。

Source Note

Title

Source URL

Problem

这篇资料解决什么失败？

Feedback Signal

它依赖什么反馈？测试、reward、工具、人类、模型自评？

Evolvable Object

它进化什么？prompt、memory、tool、workflow、agent architecture、code？

Evolution Loop

它的闭环是什么？

Evaluator

它如何判断变好了？

Safety Boundary

它如何防止坏改动？

Harness Translation

我能把它翻译成什么工程动作？

Use / Not Use

哪些适合我的场景？哪些不适合？

本章结论

自进化资料很多，但 Harness Engineering 只需要抓住一条主线：

自进化 = 反馈信号 + 可进化对象 + 改进生成 + 验证器 + 审计和回滚

反思论文告诉你如何从失败中得到语言反馈。
上下文论文告诉你如何让经验进入
playbook。

技能和实践项目告诉你如何把经验固化成可复用能力。

架构进化论文告诉你未来系统可能怎样自动搜索更好的 Agent 设计。

对于现在的个人和团队，最稳的落地路线不是直接做 Hyperagent，而是先做好三件事：

记录失败。

生成 eval。

把高价值经验提升为 memory、skill 或 tool wrapper。

这三件事做扎实，Harness 就已经开始自进化。

33. 推荐阅读：按进化主线学习 Harness

本章学习目标

推荐阅读不是堆链接，而是给你一条学习路径。不同资料解决不同问题，不要一上来全读。

阅读顺序建议

阅读顺序建议按“先工程系统，后进化机制，再看场景”的顺序走。

第一轮：建立 Harness 基础概念。

1. OpenAI Harness Engineering。
2. Martin Fowler Harness Engineering。
3. Agents-Zone Playbook。

第二轮：理解 Harness 的进化主线。

1. A Survey of Self-Evolving Agents。
2. Beyond Individual Intelligence。
3. Agentic Context Engineering。
4. Externalization in LLM Agents。

第三轮：理解反思、反馈和 eval。

1. Reflexion。
2. Self-Refine。
3. CRITIC。
4. OpenAI Eval Skills。
5. LangChain Agent Harness。

第四轮：理解长任务和编码场景。

1. Anthropic long-running agents。
2. Anthropic Claude Code harness。
3. Cursor Codex model harness。
4. ReVeal。

第五轮：理解系统约束和实践项目。

1. Yage prompt caching。
2. arXiv externalization review。
3. wow-harness。
4. self-improving-agent。
5. Evolver。

第六轮：理解高级架构进化。

1. Promptbreeder。
2. Automated Design of Agentic Systems。
3. STOP。
4. Darwin Gödel Machine。

5. Hyperagents。

6. AlphaEvolve。

阅读时要带的问题

每读一篇，记录：

- 这篇定义的 Harness 是什么。
- 它主要解决哪类失败。
- 它用了哪些组件。
- 它如何验证有效。
- 它有什么局限。
- 我能借走什么到自己的任务里。

一手资料

- [OpenAI: Harness engineering](#)
- [Anthropic: Harness design for long-running agents](#)
- [Anthropic: Effective harnesses for Claude Code](#)
- [Cursor: Codex model harness](#)
- [OpenAI Developers: Eval skills](#)

方法论

- [Martin Fowler: Harness Engineering](#)

- [LangChain: The Anatomy of an Agent Harness](#)
- [LangChain: Improving Deep Agents with Harness Engineering](#)
- [Agents-Zone: Harness Engineering Playbook](#)
- [arXiv: Externalization in LLM Agents](#)

中文资料

- [Yage: Prompt caching as Harness constraint](#)
- [Yage: CLI-Anything HARNESS.md 方法论拆解](#)
- [Yage: Harness Engineering 的需求侧分析](#)
- [微信文章: Harness Engineering 的四大问题](#)

开源项目

- [NatureBlueeee/wow-harness](#)
- [Agents-Zone/harness-engineering-playbook](#)
- [browser-use/browser-harness](#)
- [walkinglabs/awesome-harness-engineering](#)

自进化专题

如果你关心 Harness 如何从失败中持续变强，先读第 6 章建立工程模型，再读第 32 章按“综述、反思、自修正、上下文进化、技能进化、架构进化”的顺序展开资料。

最重要的三篇是：

- [A Survey of Self-Evolving Agents](#)
 - [Beyond Individual Intelligence](#)
 - [Agentic Context Engineering](#)
-

结语

Harness Engineering 的核心不是追逐新概念，而是把 Agent 工作变成一个能从证据中持续改进的工程系统。

你可以用三个问题检验自己是否真正理解了 Harness：

1. 如果 Agent 做错了，我能知道错在哪里吗？
2. 如果同类任务再来一次，系统会比上次更好吗？
3. 如果我放手让 Agent 执行更多步骤，风险是否仍在可控范围内？

如果答案是肯定的，你就在做 Harness Engineering。

如果答案是否定的，你还只是在使用一个会调用工具模型。

真正的 Harness 不一定复杂，但一定有闭环。它让 Agent 不只是“能做事”，而是“在可控系统里持续做得更好”。

作者联系

作者：大铭

邮箱：yinwm@outlook.com



大铭 
中国大陆



扫一扫上面的二维码图案，加我为朋友。